

Deterministic Finite Automata

And Regular Languages

Deterministic Finite Automata

Finite Automata

- A finite automaton is a model that has a finite set of states (represented in the figure by circles) and its control moves from one state to another state in response to external inputs (represented by arrows).
- It is an abstract machine that is used to recognize patterns in strings of symbols. Finite automata are widely used in computer science for pattern matching, lexical analysis and parsing among other applications.

Simple Automaton

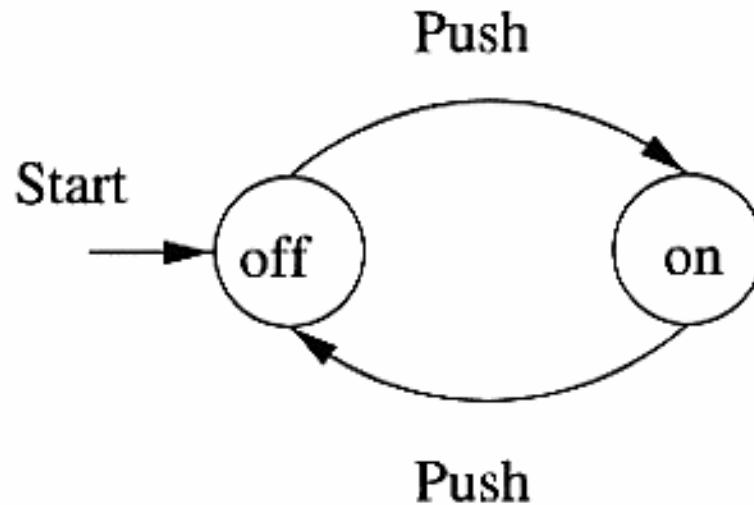


Figure 1.1: A finite automaton modeling an on/off switch

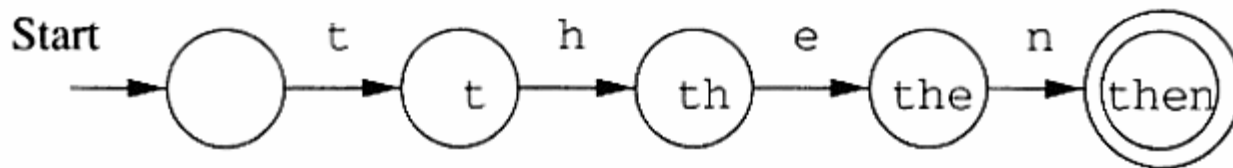


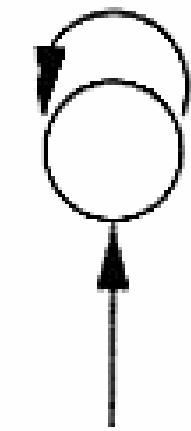
Figure 1.2: A finite automaton modeling recognition of **then**

Complex Automaton

1. The customer may decide to *pay*. That is, the customer sends the money to the store.
2. The customer may decide to *cancel*. The money is sent to the bank with a message that the value of the money is to be added to the customer's bank account.
3. The store may *ship* goods to the customer.
4. The store may *redeem* the money. That is, the money is sent to the bank with a request that its value be given to the store.
5. The bank may *transfer* the money by creating a new, suitably encrypted money file and sending it to the store.

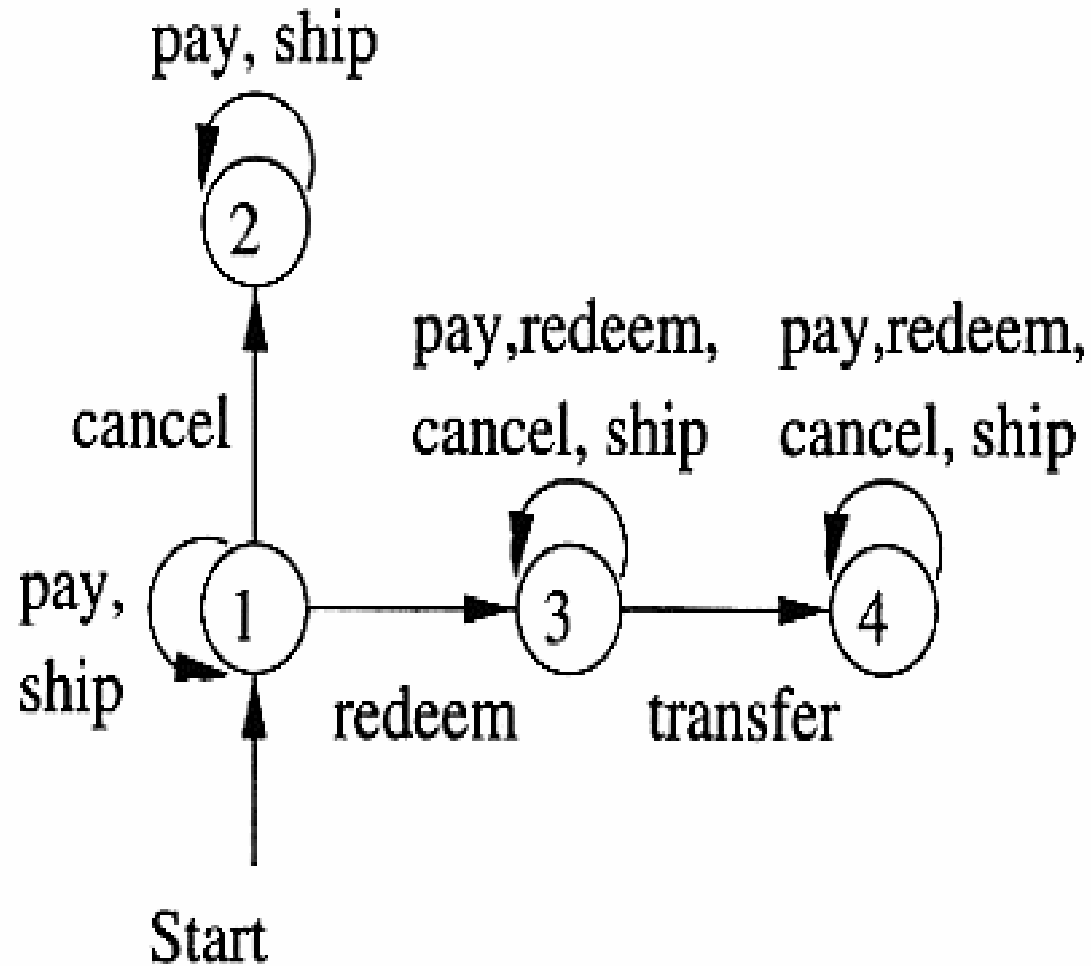
Complex Automaton (cont..)

ship, redeem, transfer,
pay, cancel



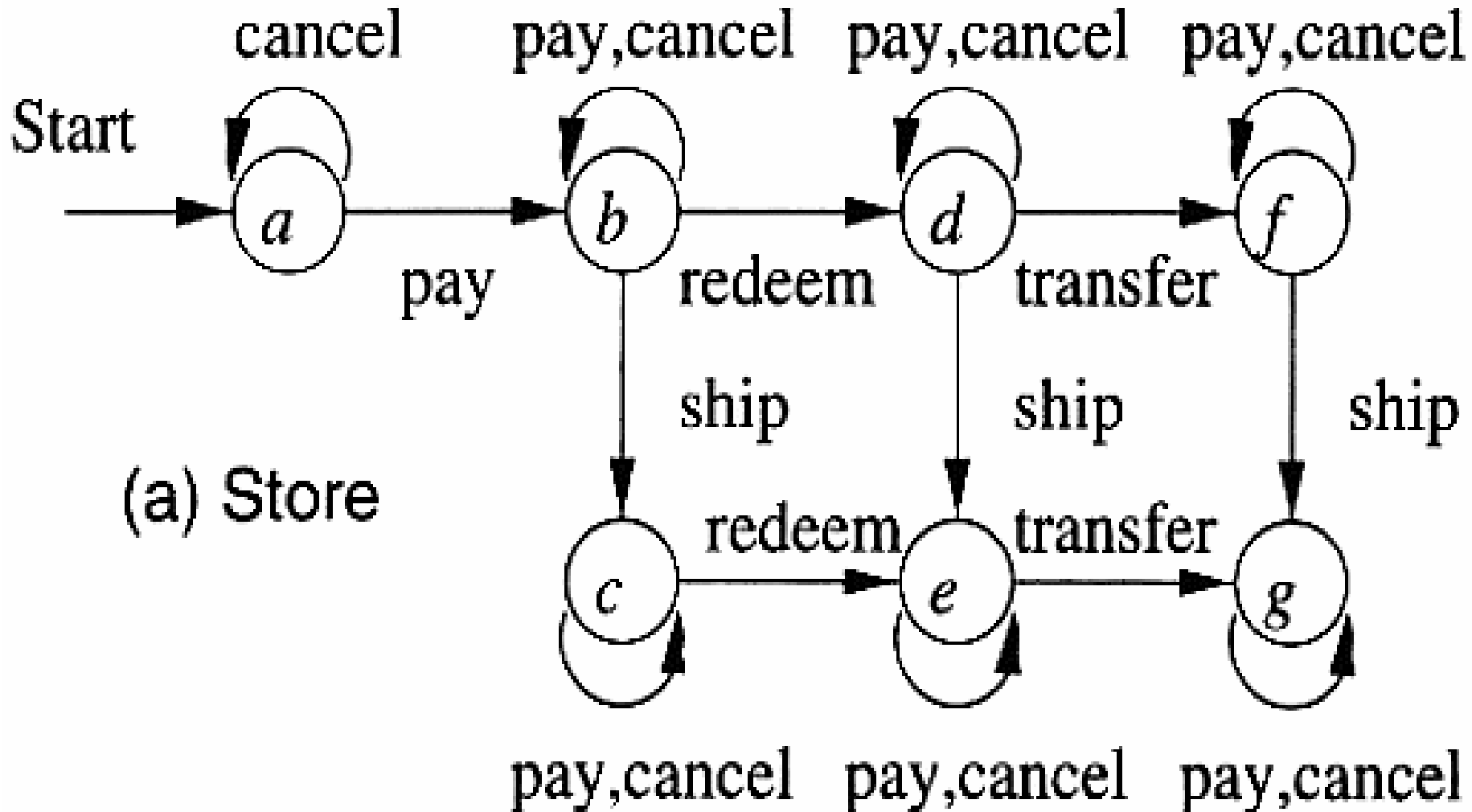
Start

(b) Customer



(c) Bank

Complex Automaton (cont..)



Complex Automaton (cont..)

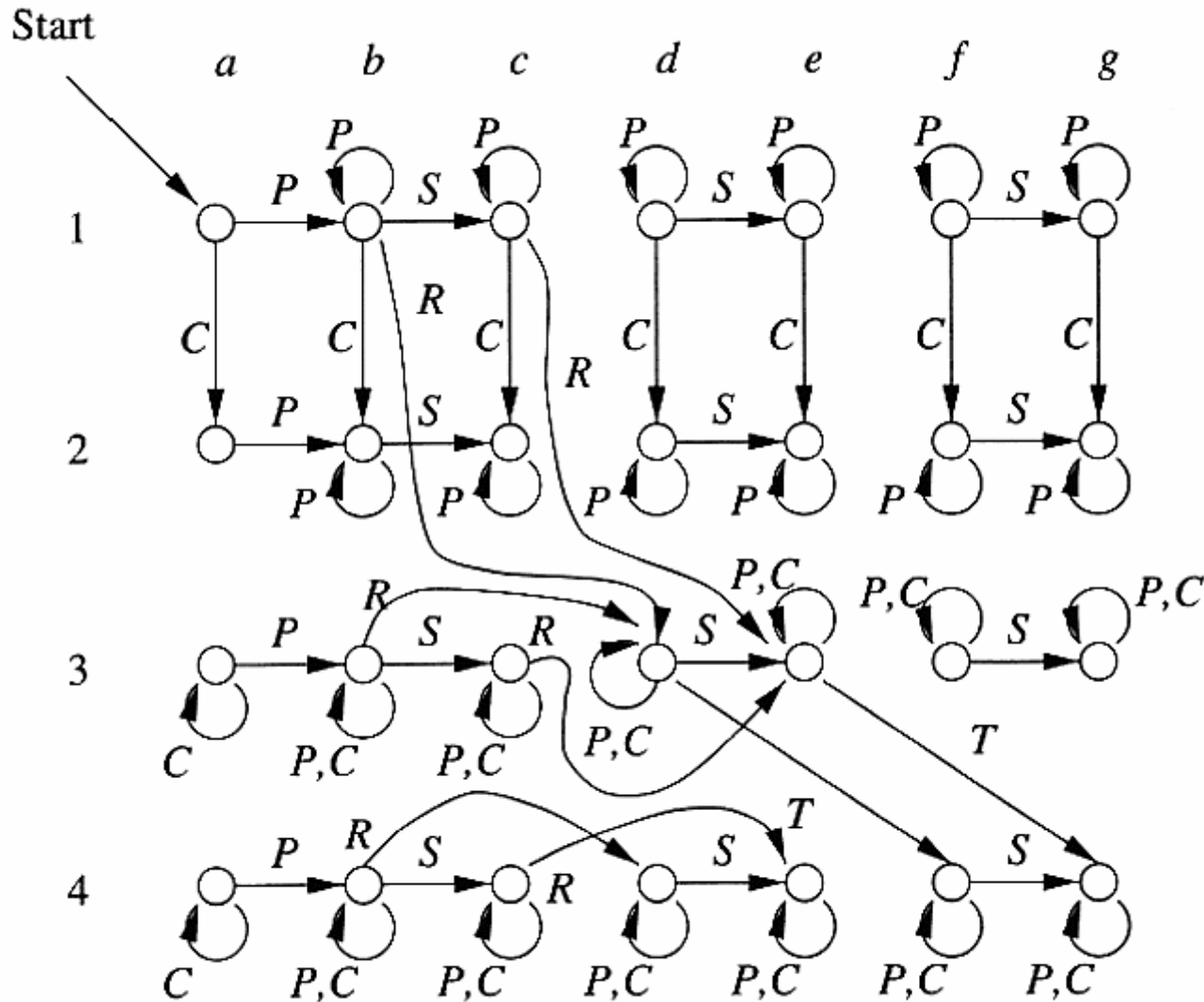
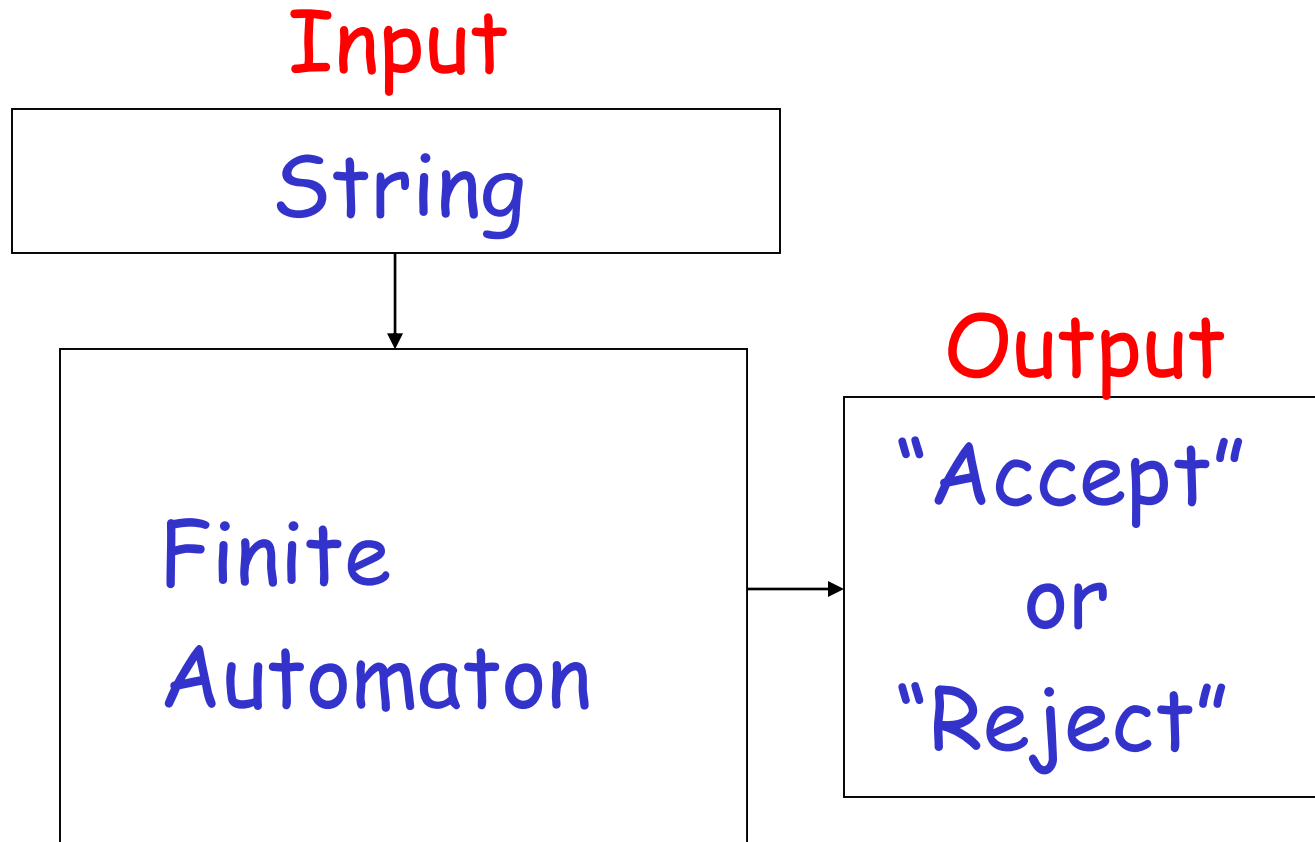


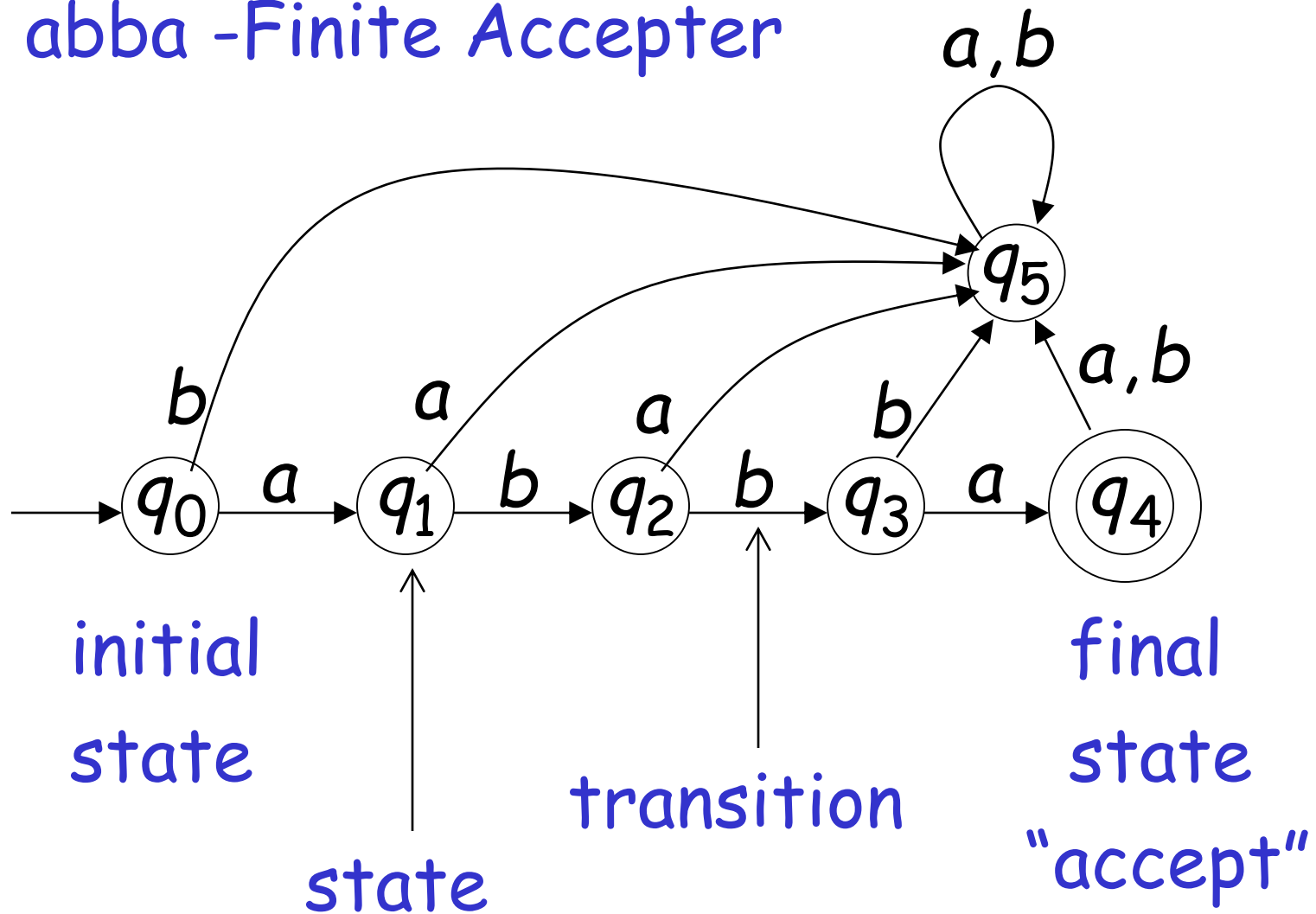
Figure 2.3: The product automaton for the store and bank

Finite Acceptor

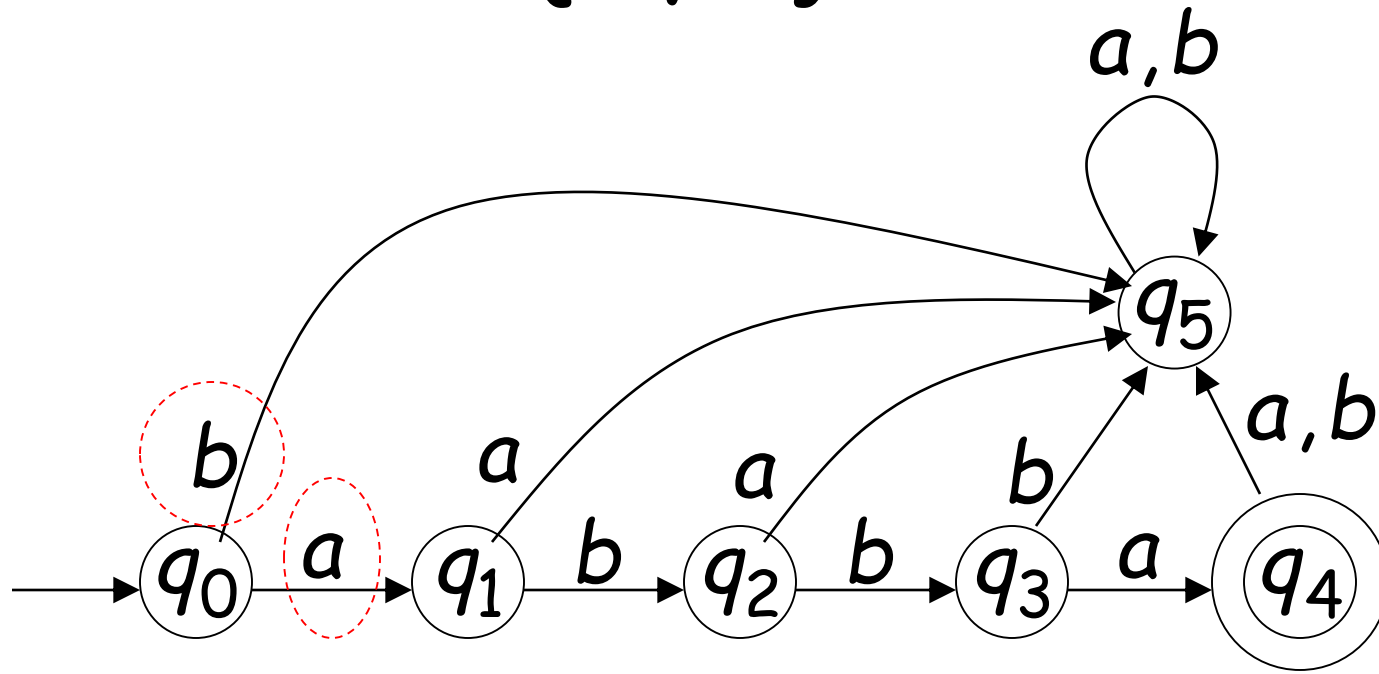


Transition Graph

abba -Finite Acceptor

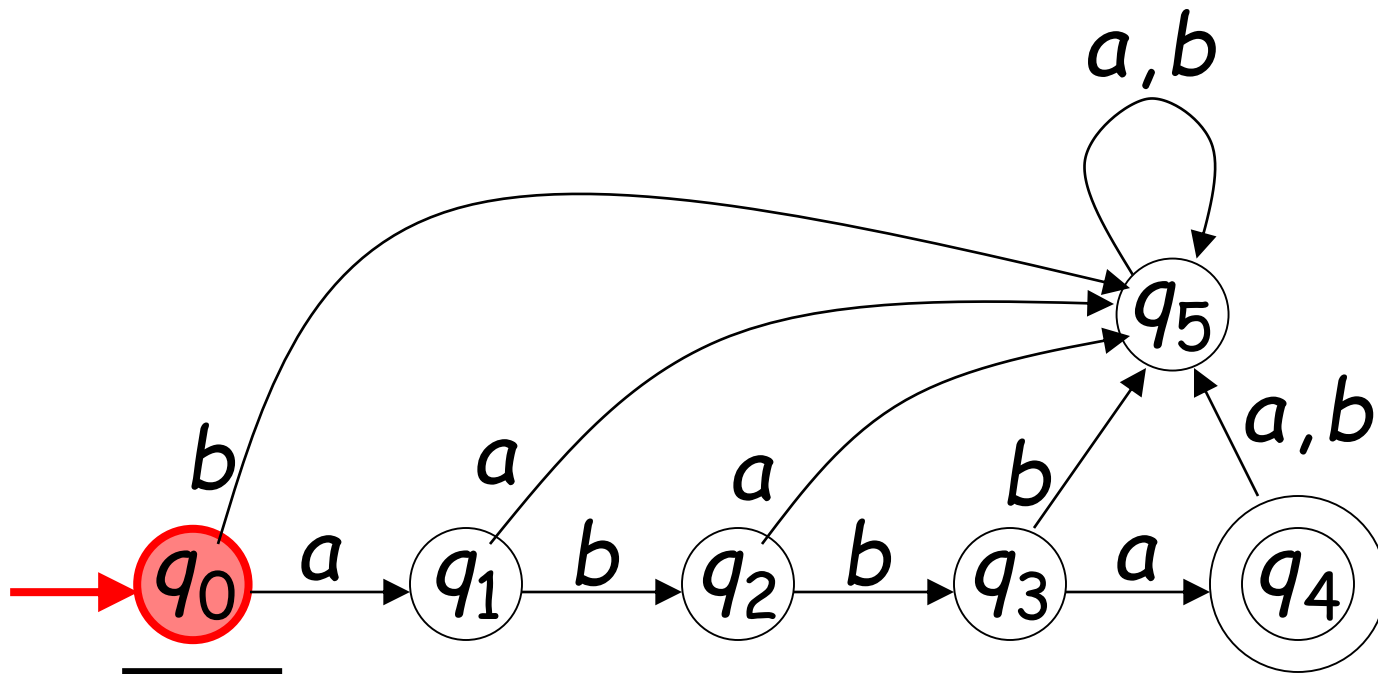


Alphabet $\Sigma = \{a, b\}$

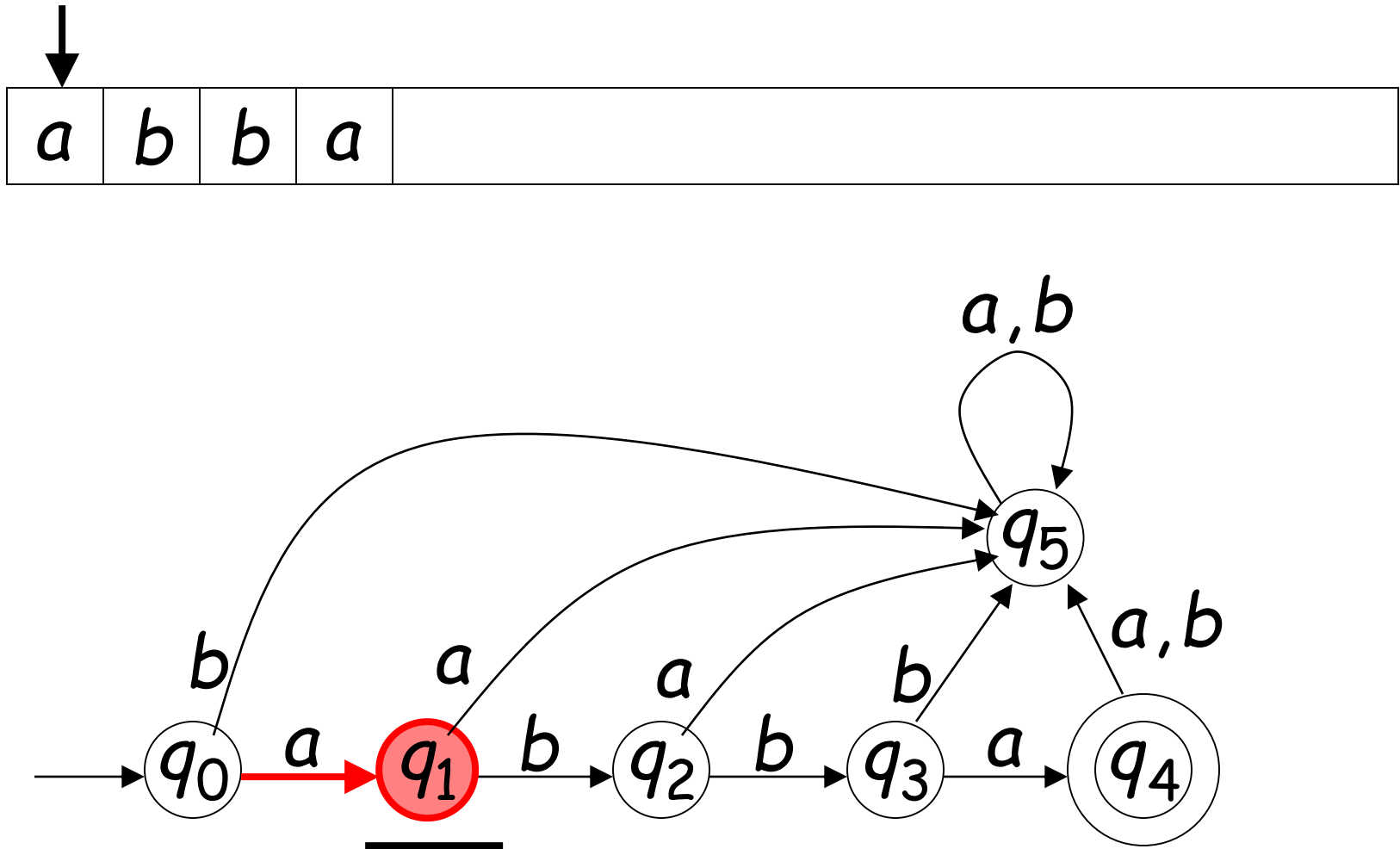


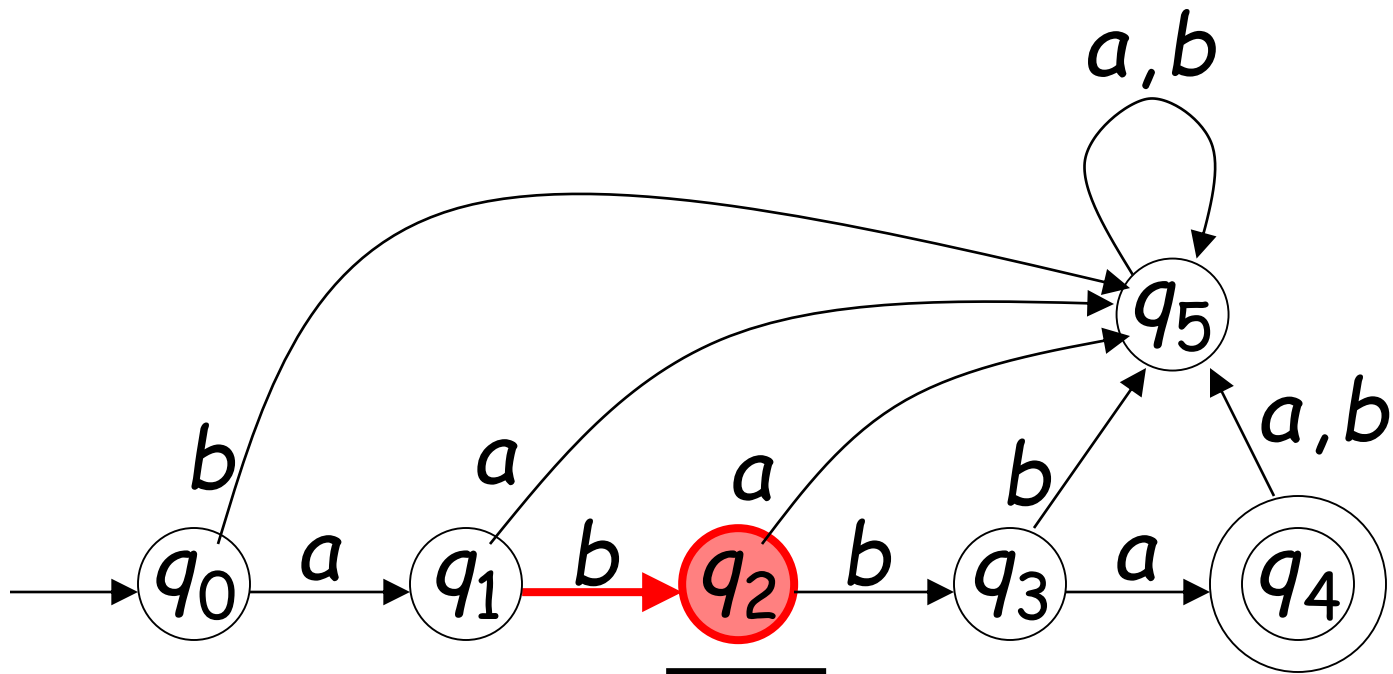
For every state, there is a transition for every symbol in the alphabet

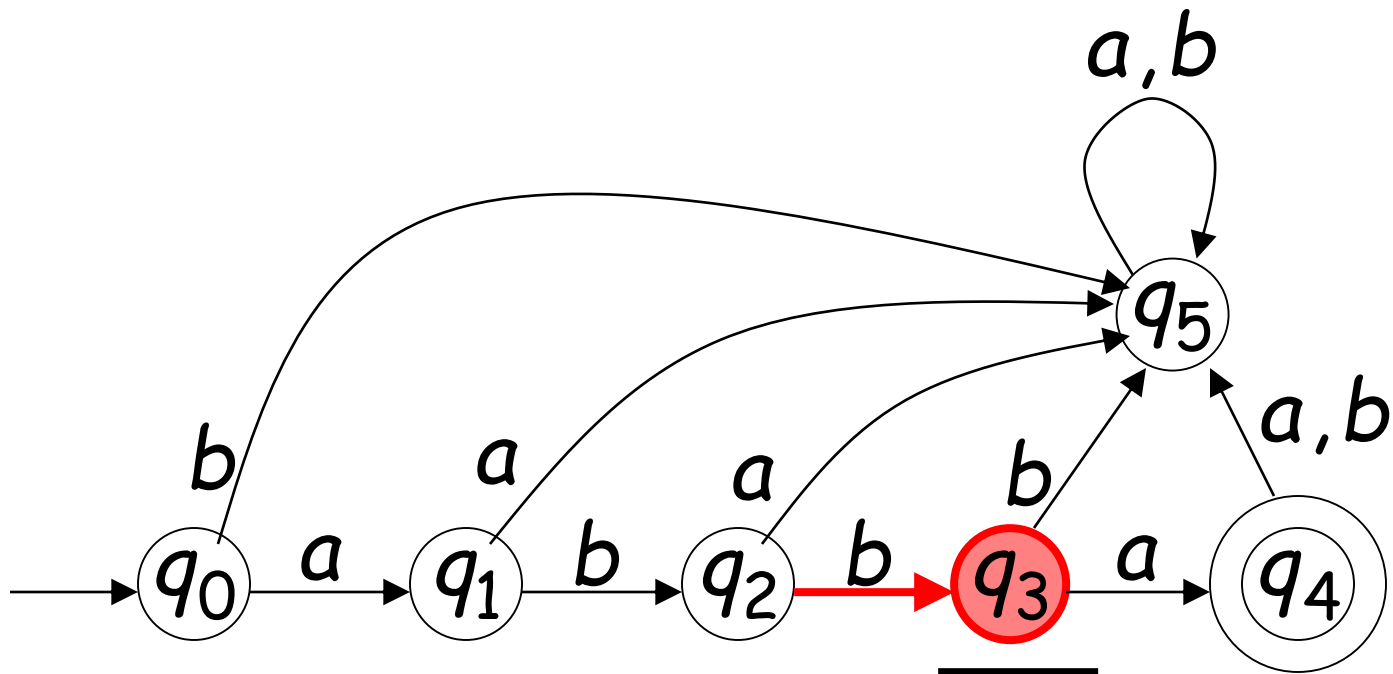
Initial Configuration

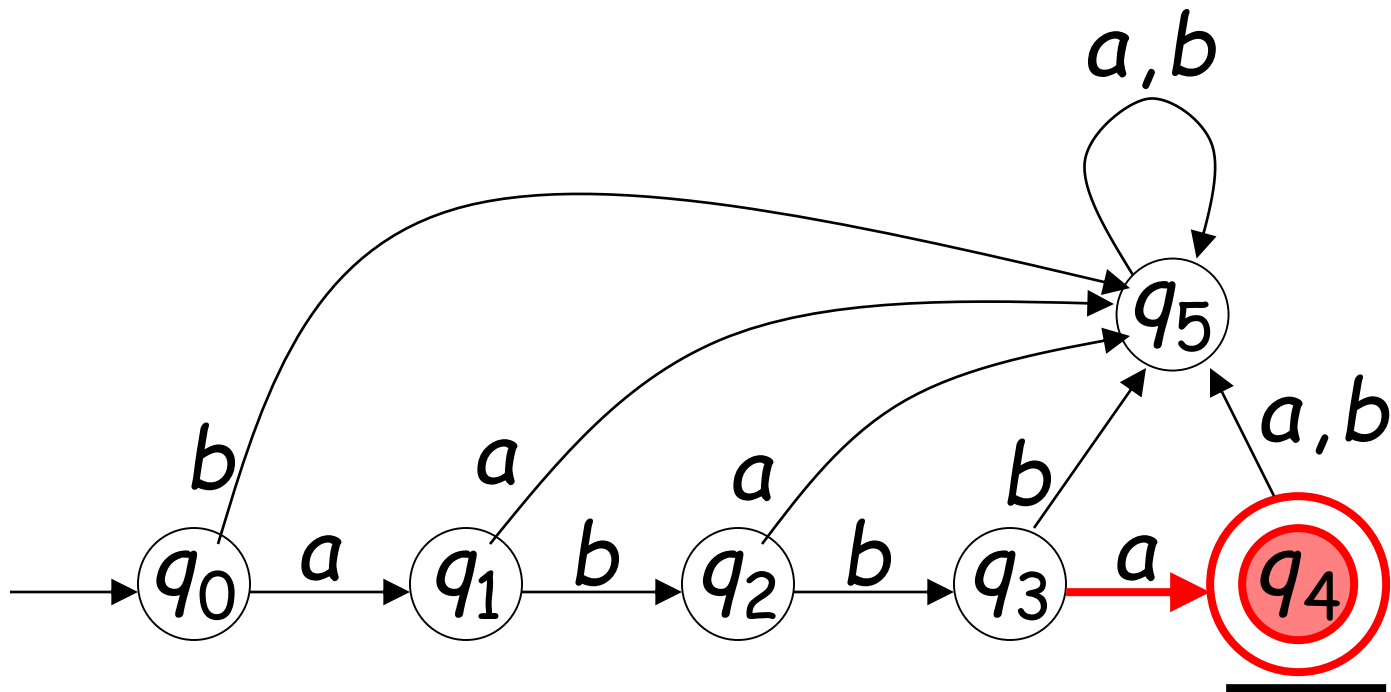


Reading the Input

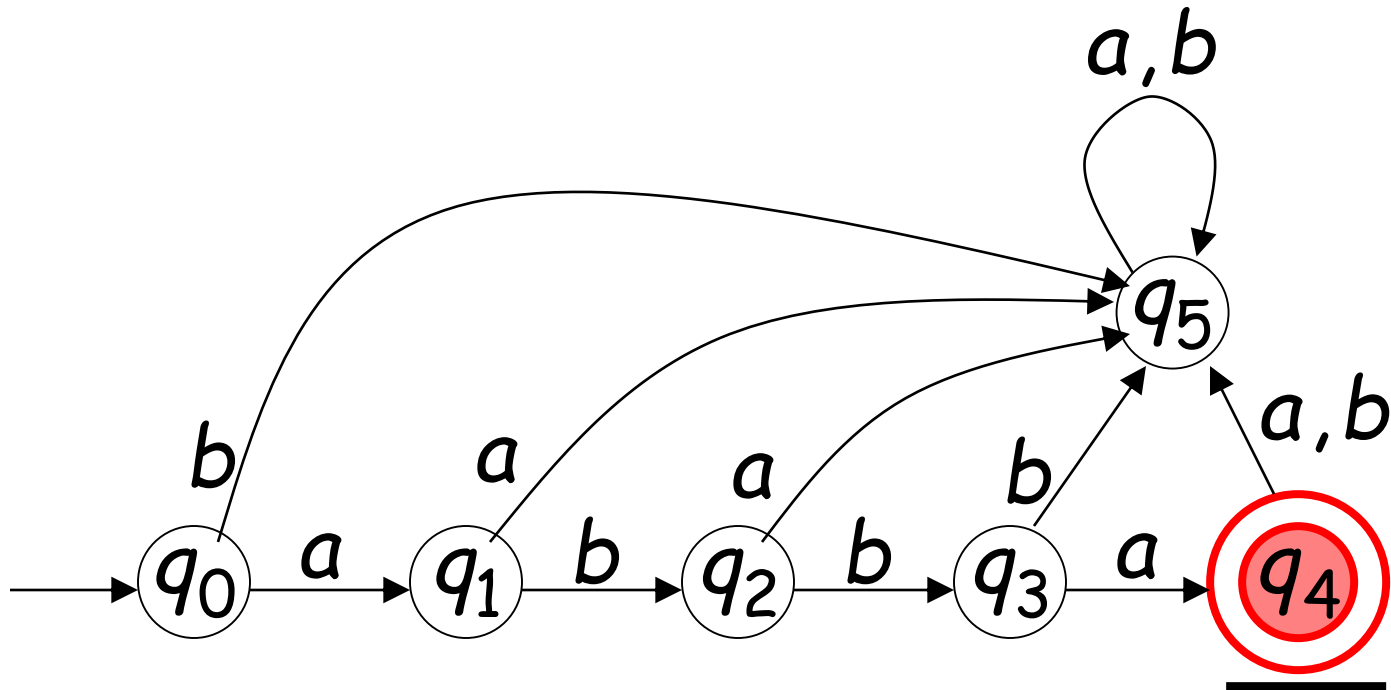
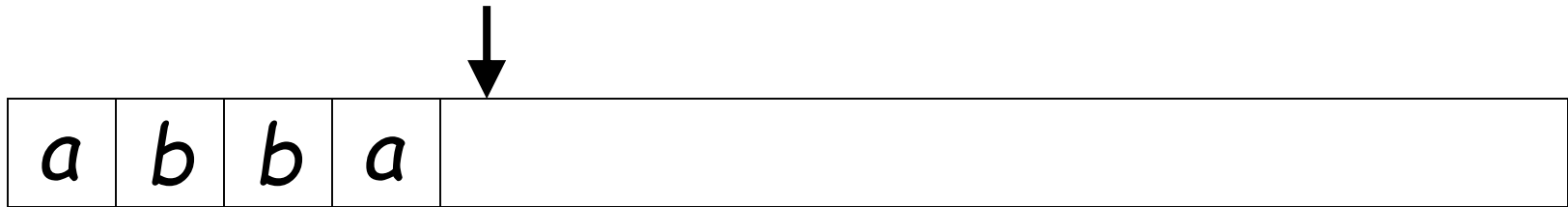






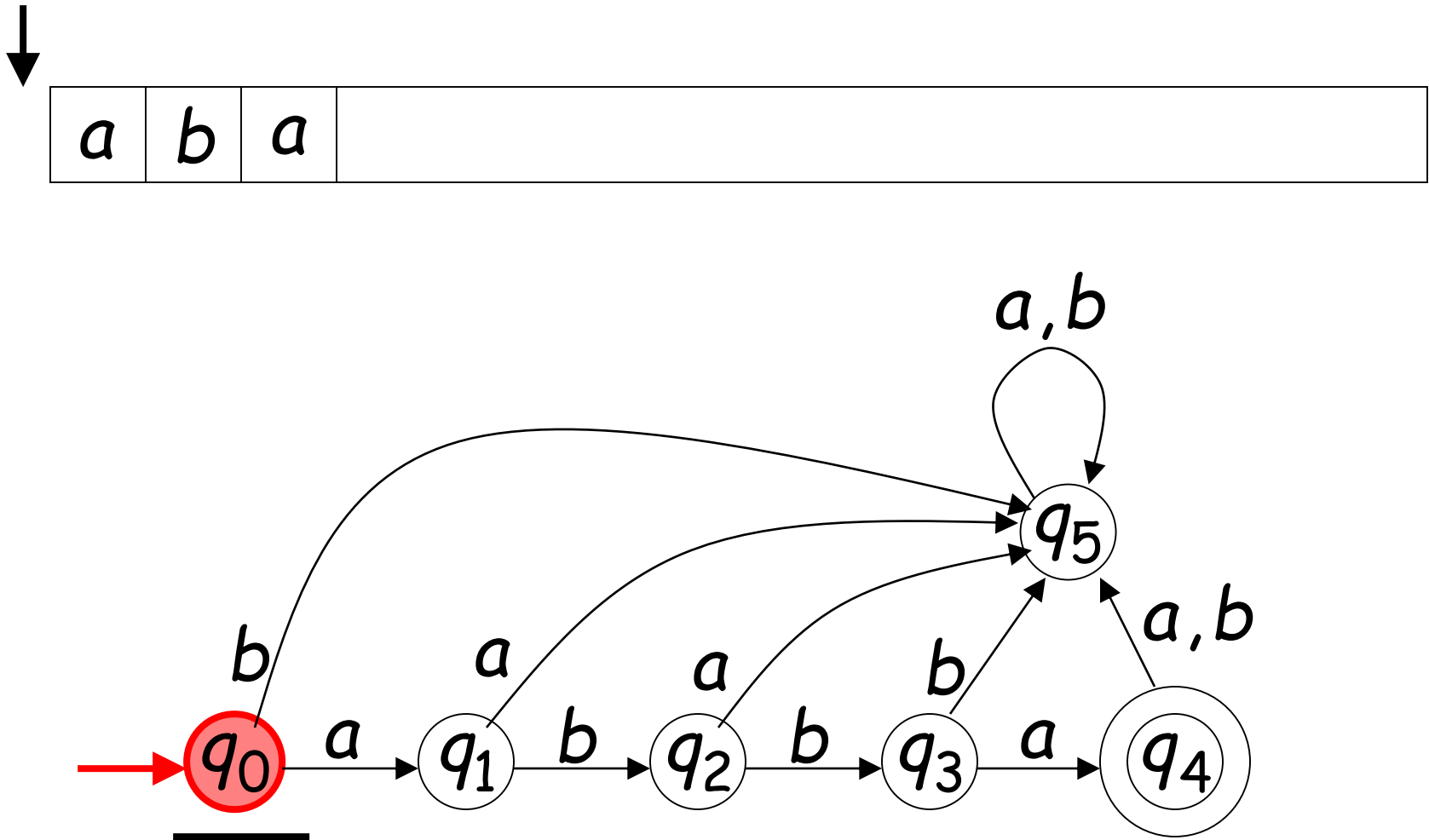


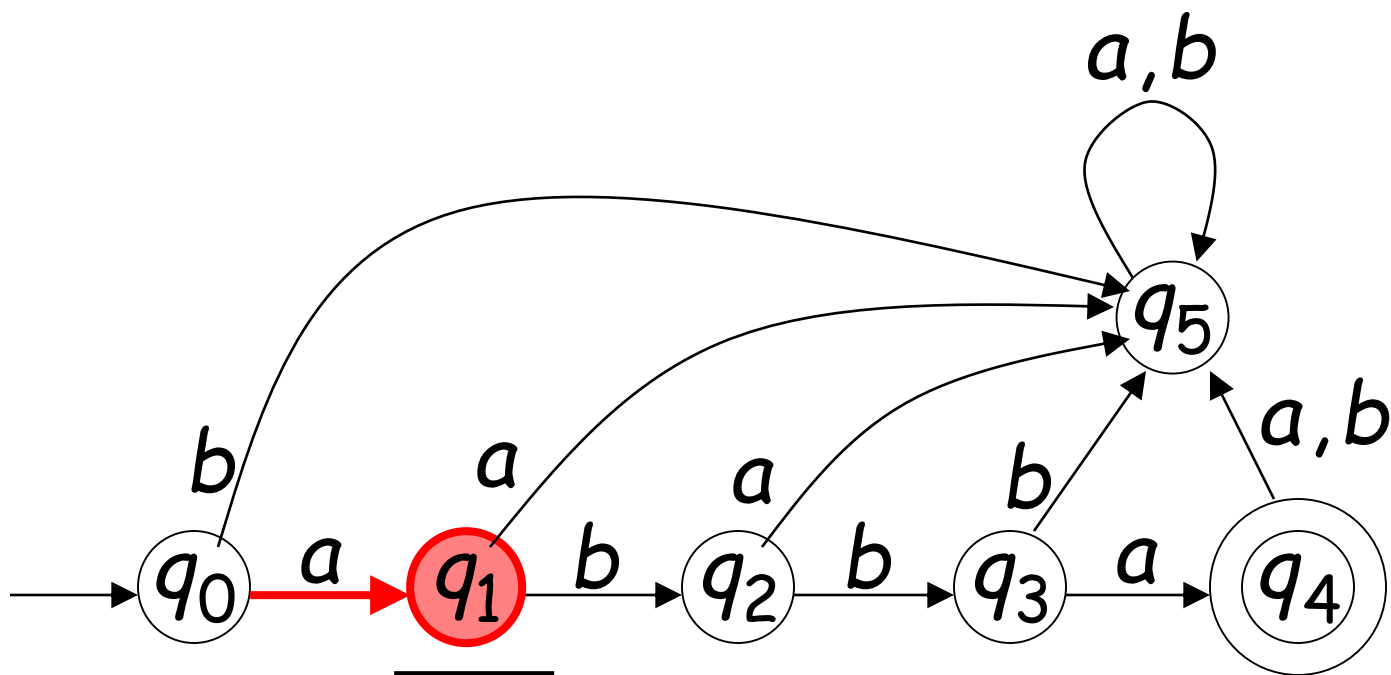
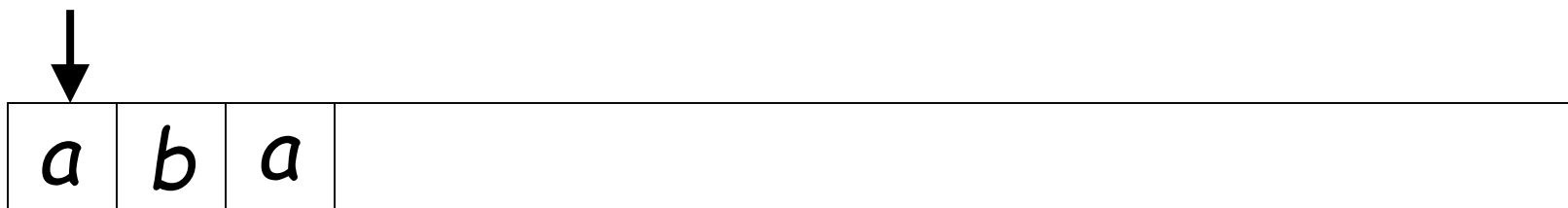
Input finished

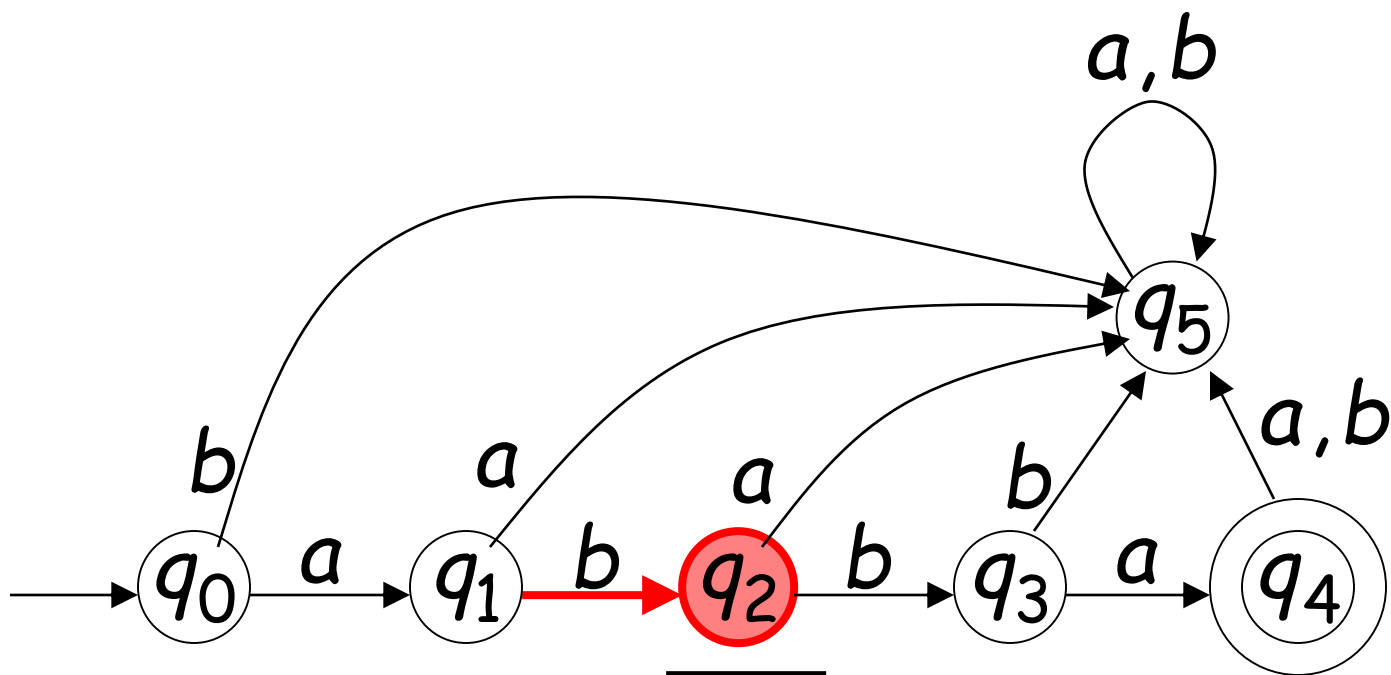
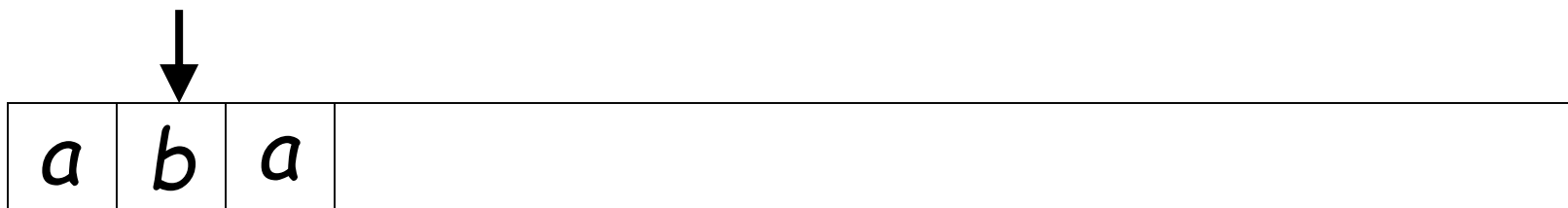


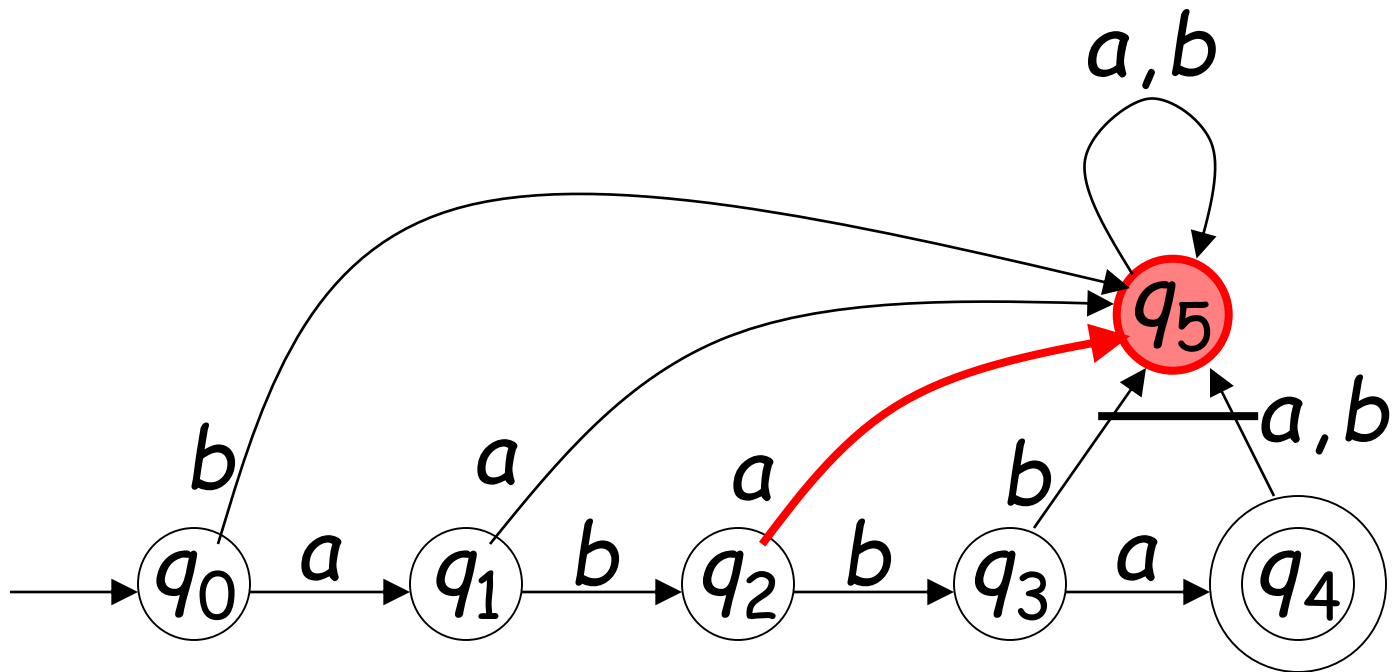
Output: "accept"

Rejection

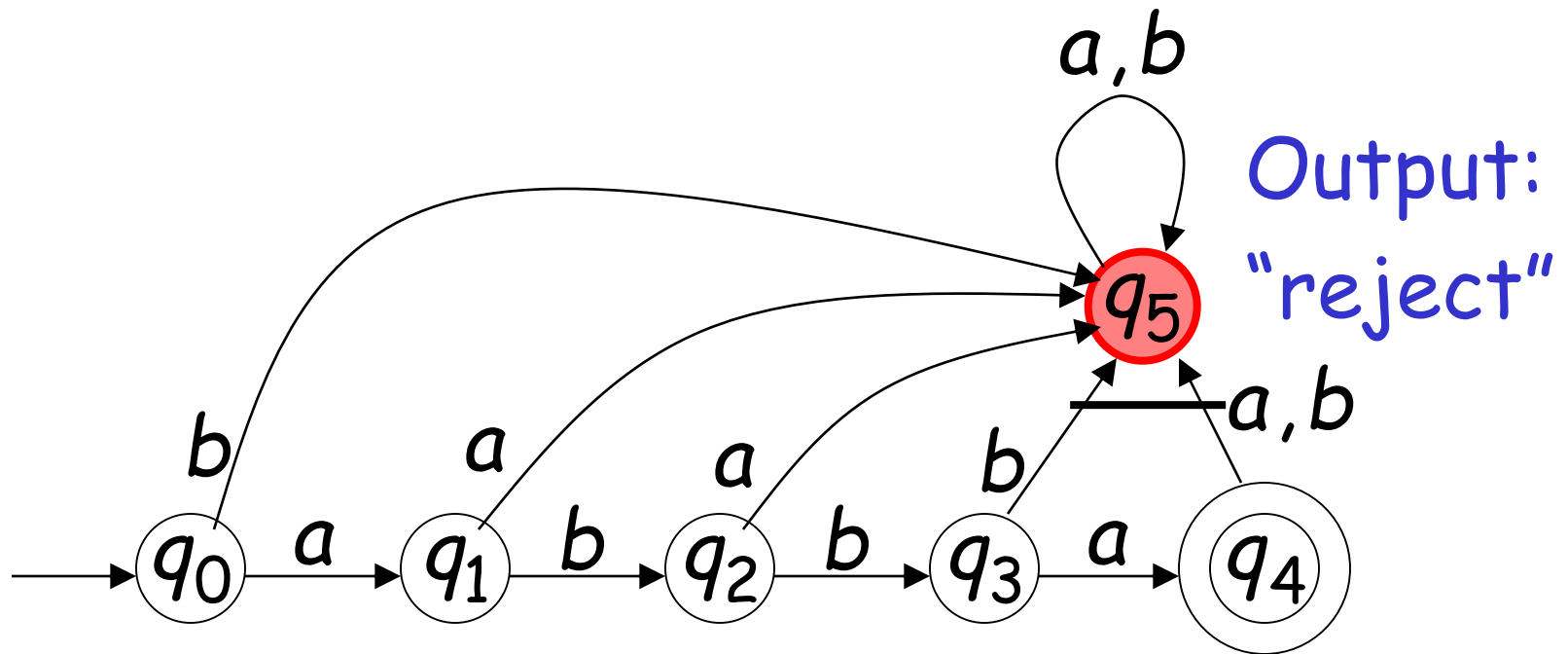
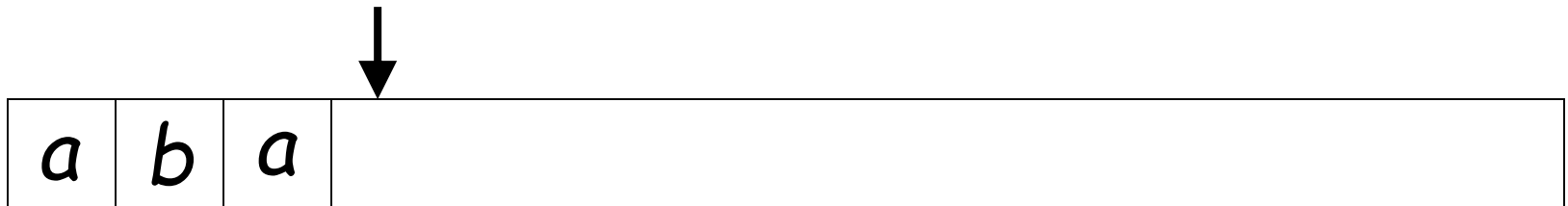




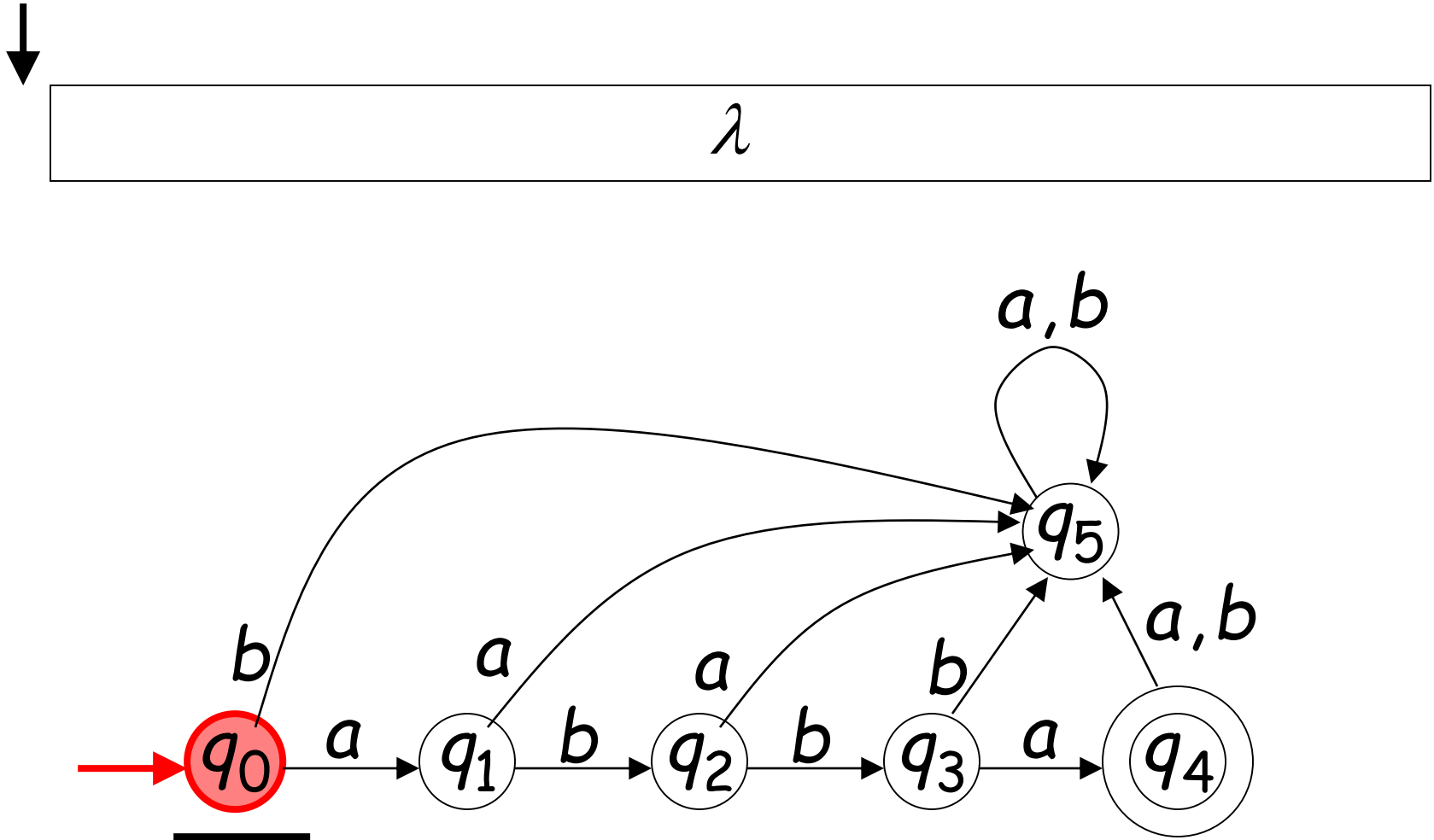




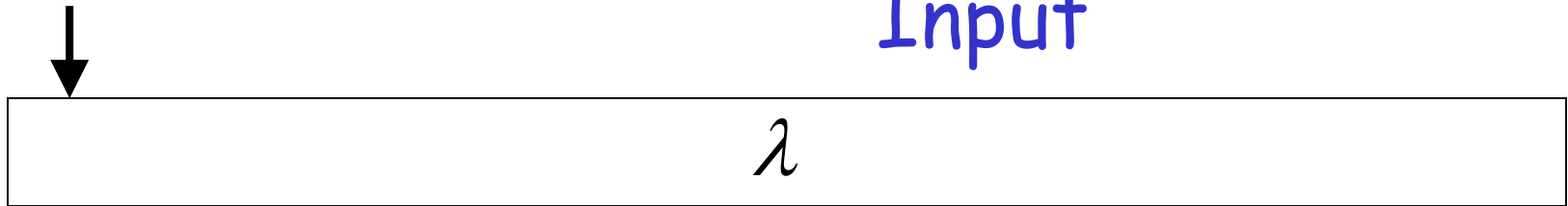
Input finished



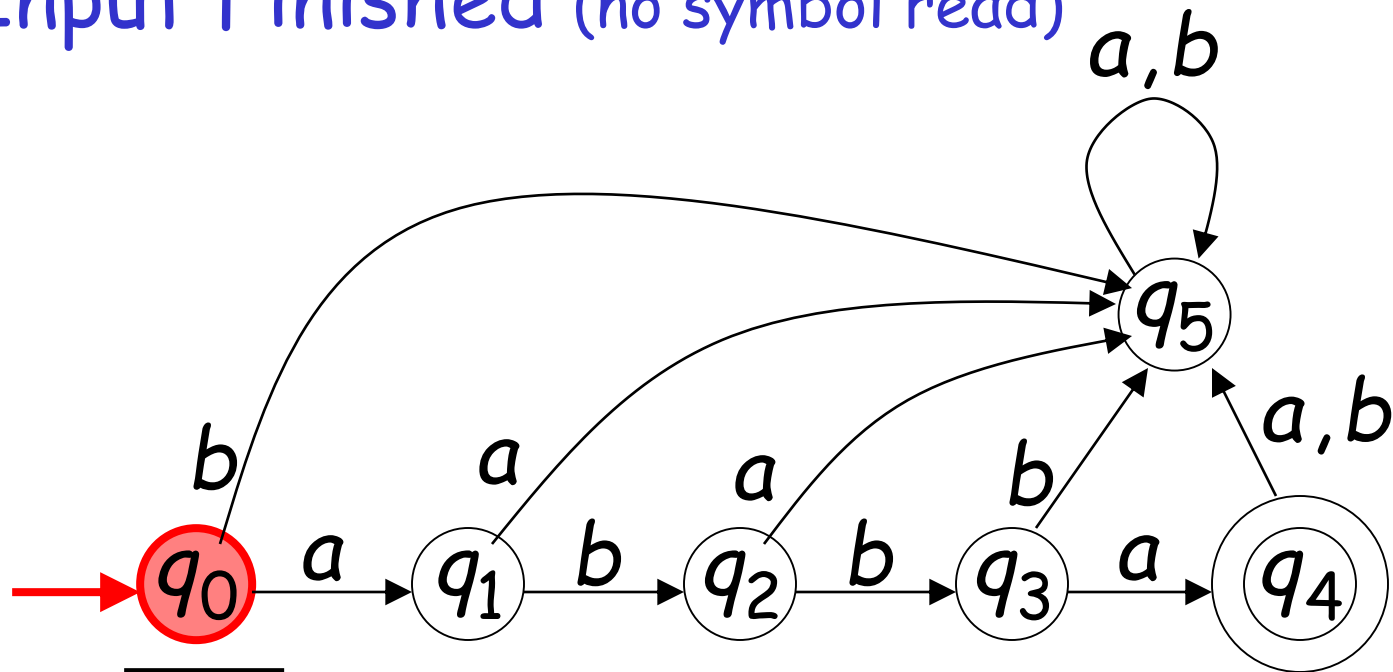
Another Rejection



Input

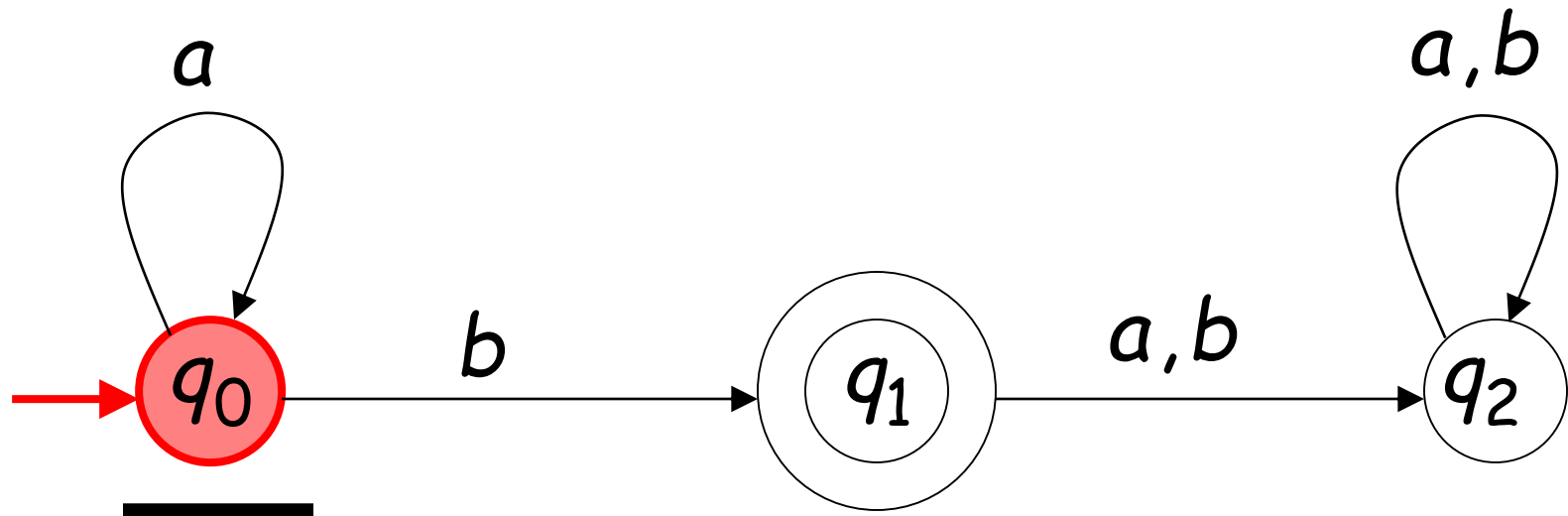
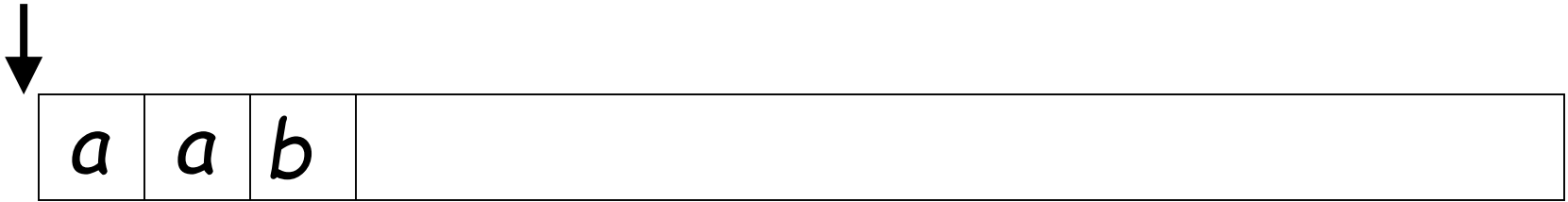


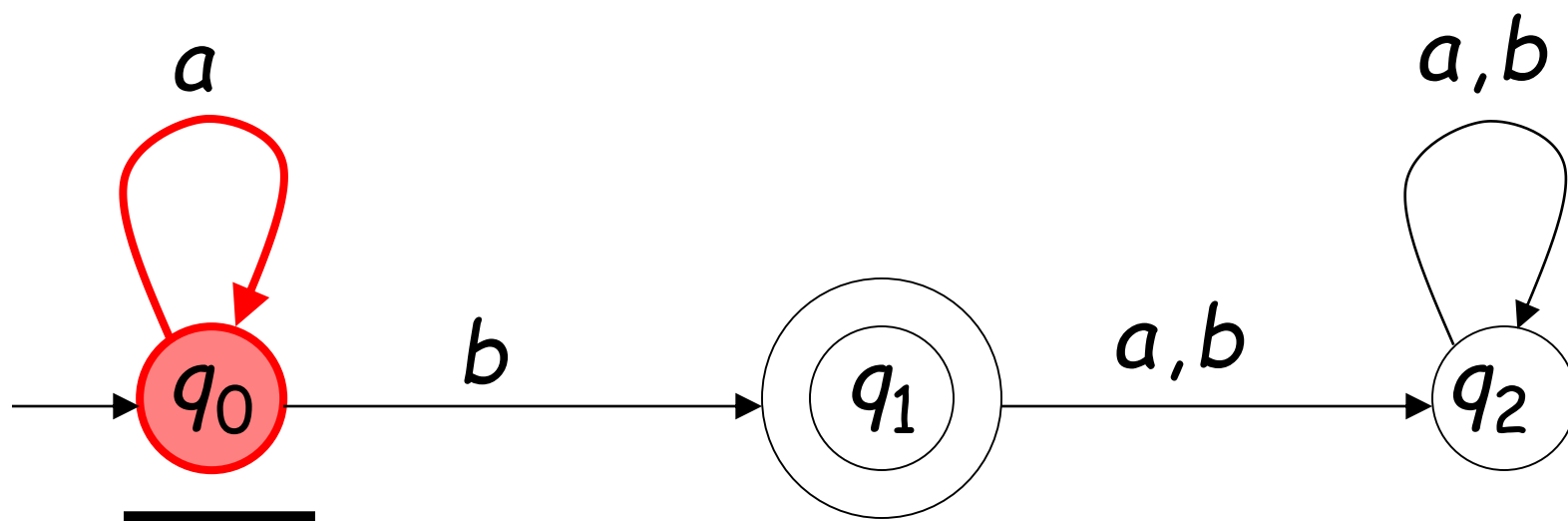
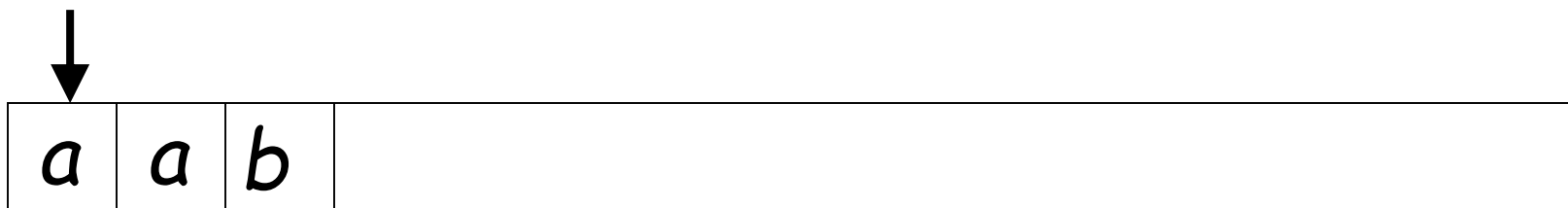
Input Finished (no symbol read)

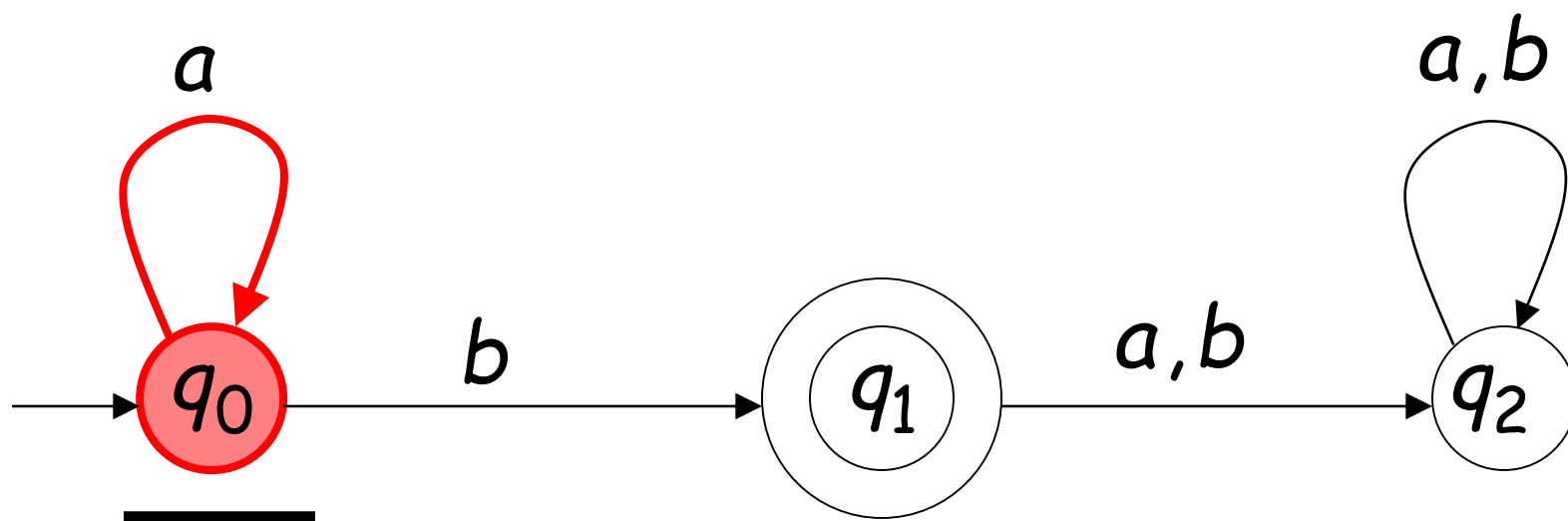
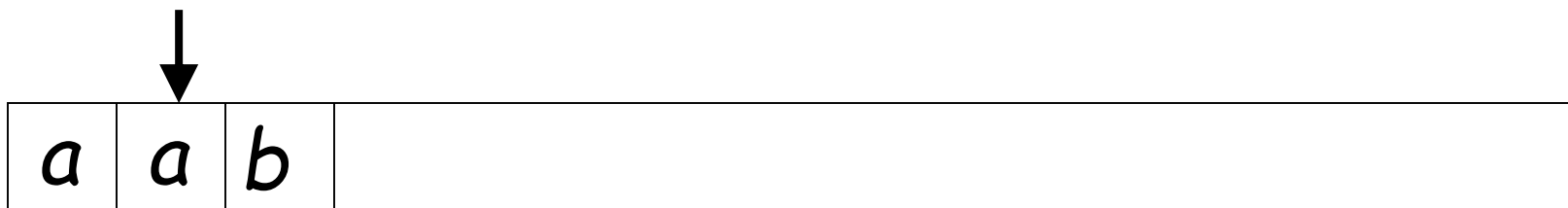


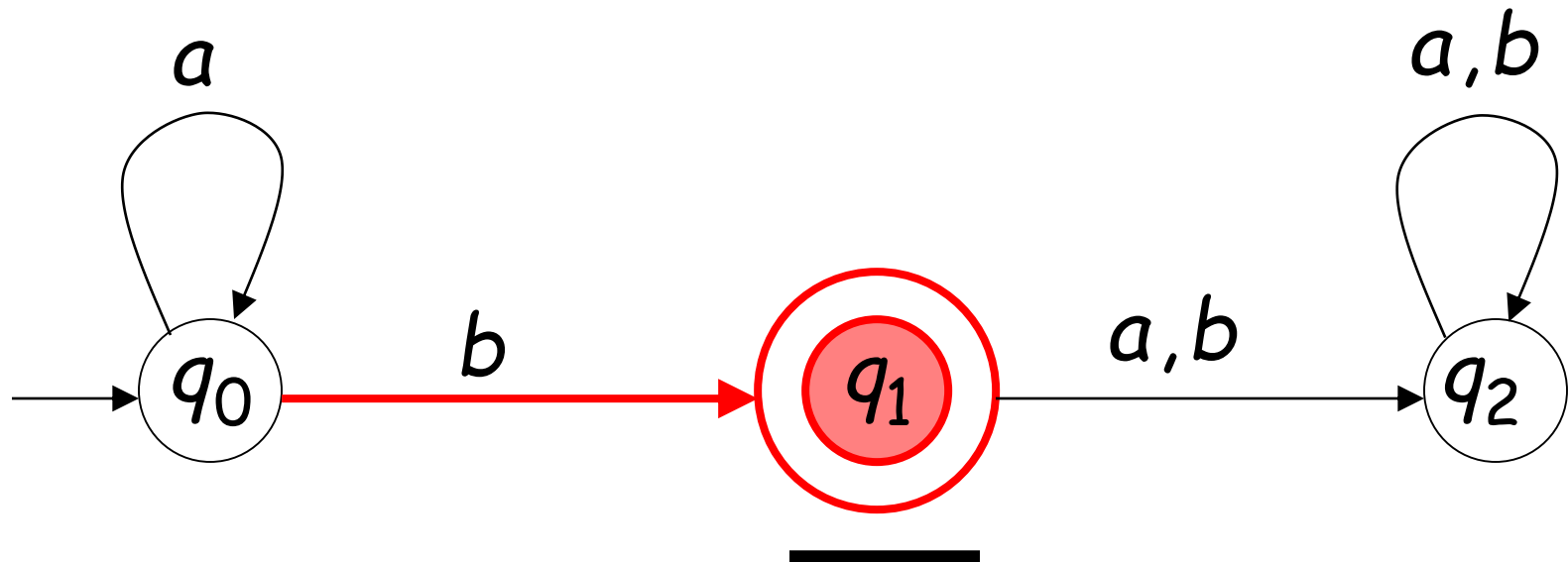
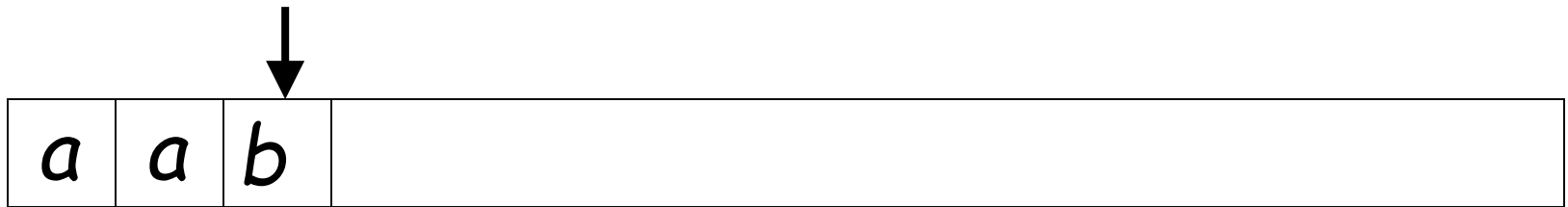
Output:
"reject"

Another Example

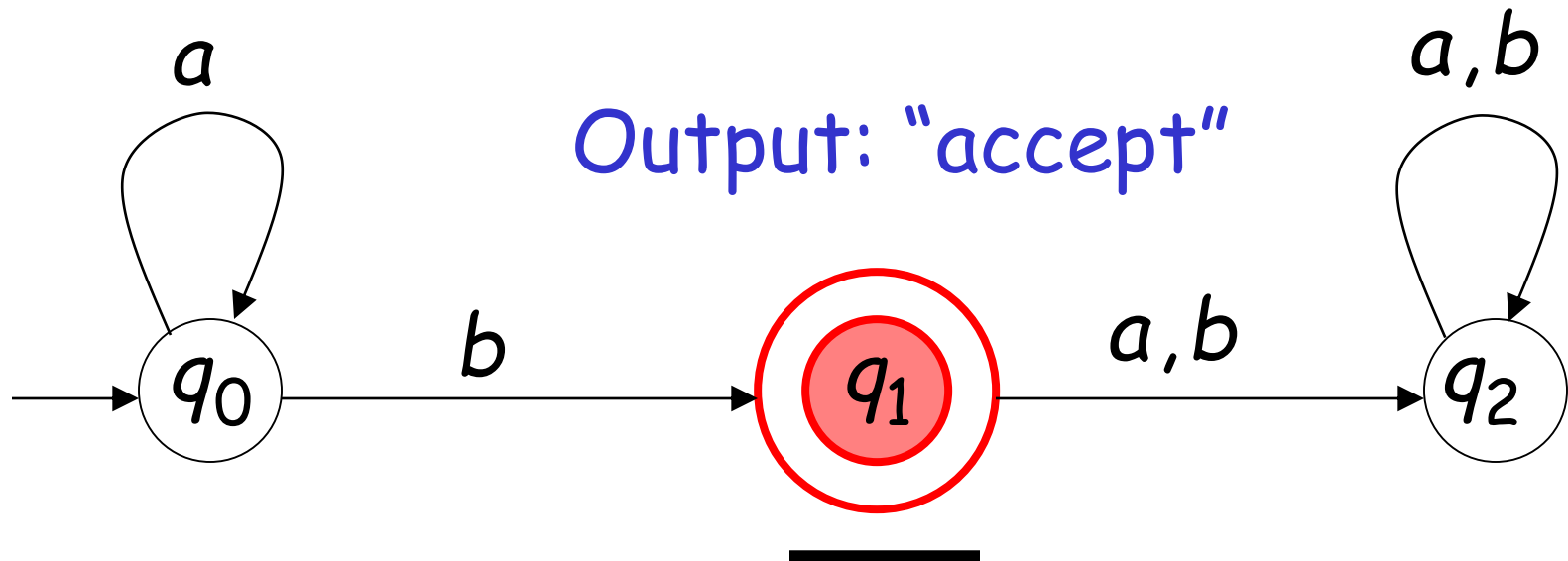




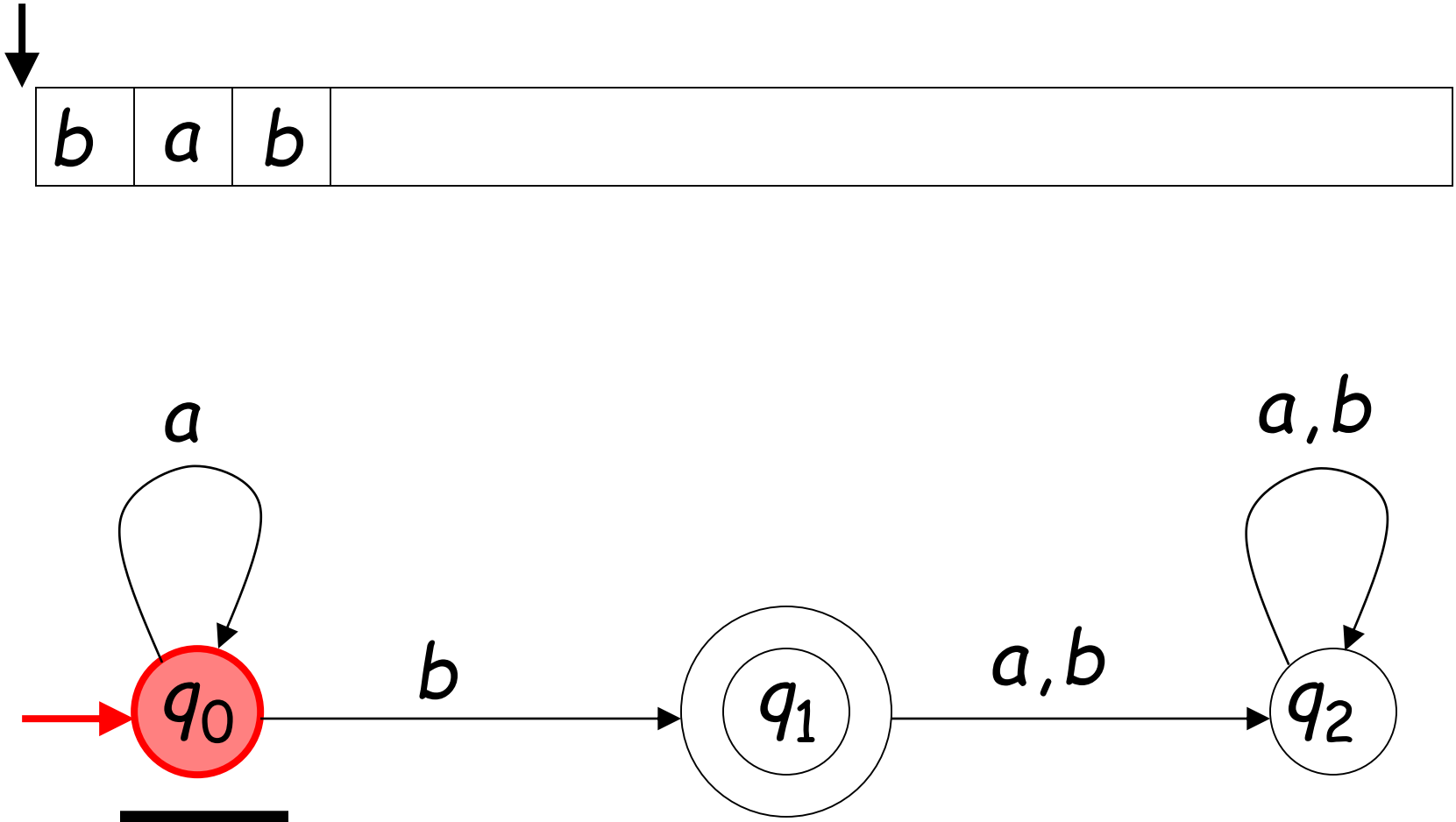


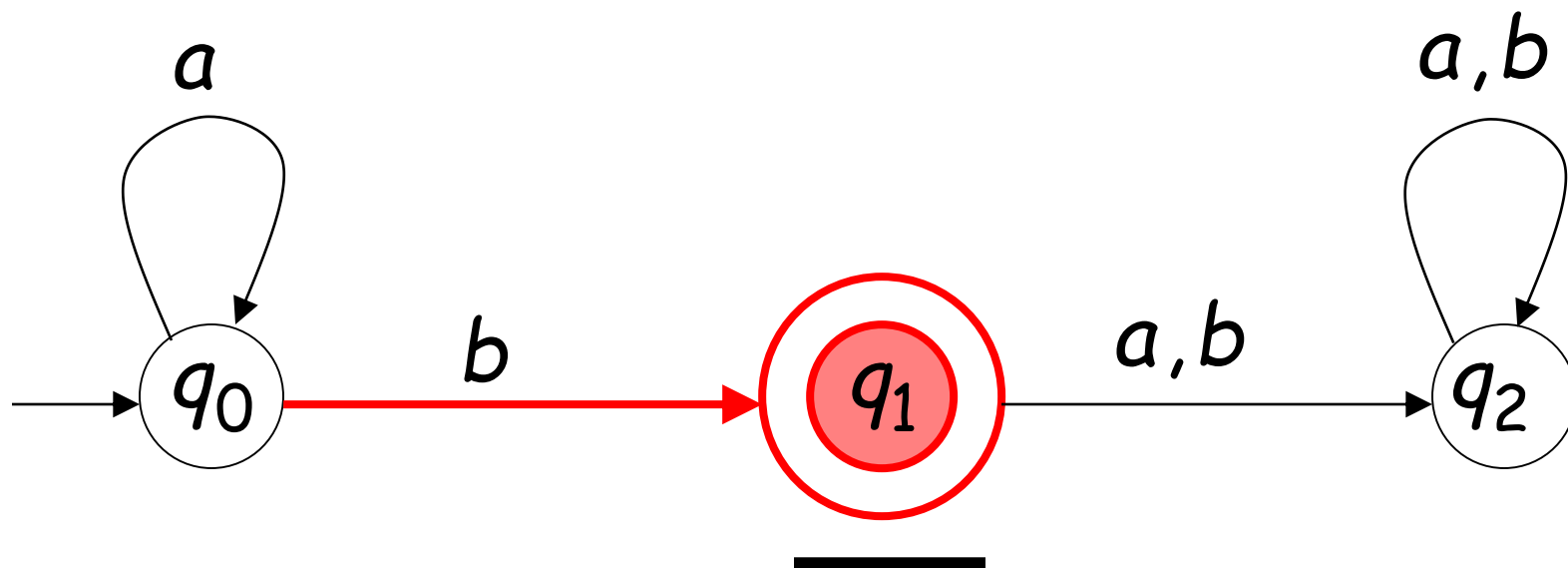


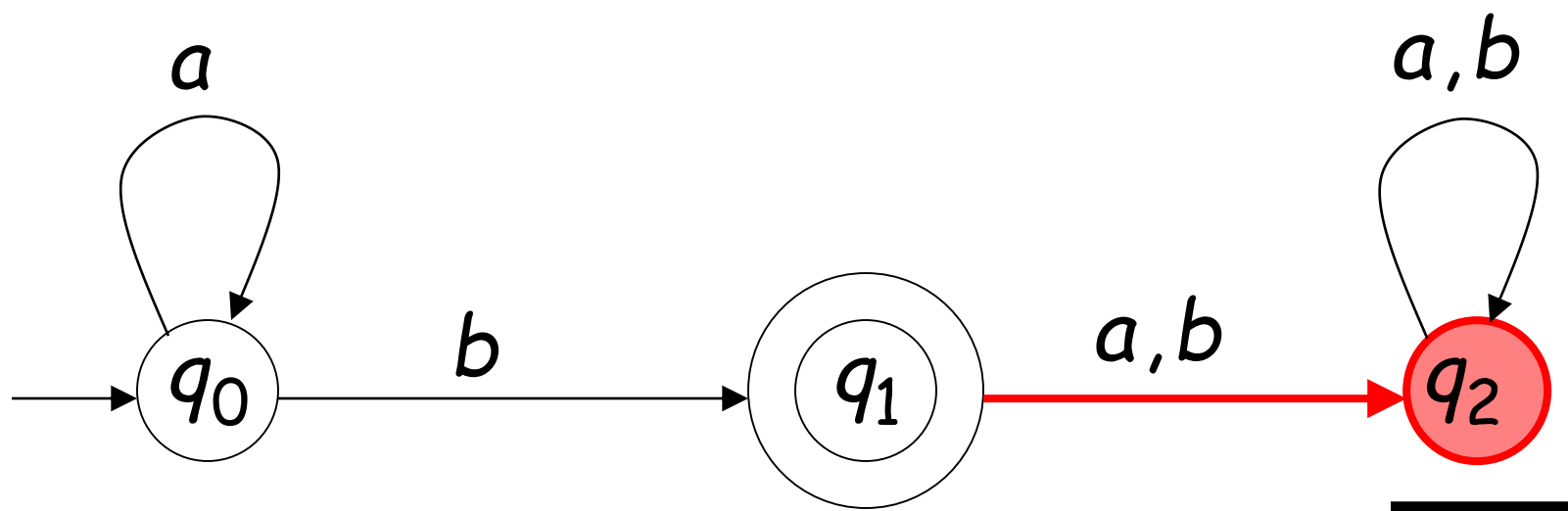
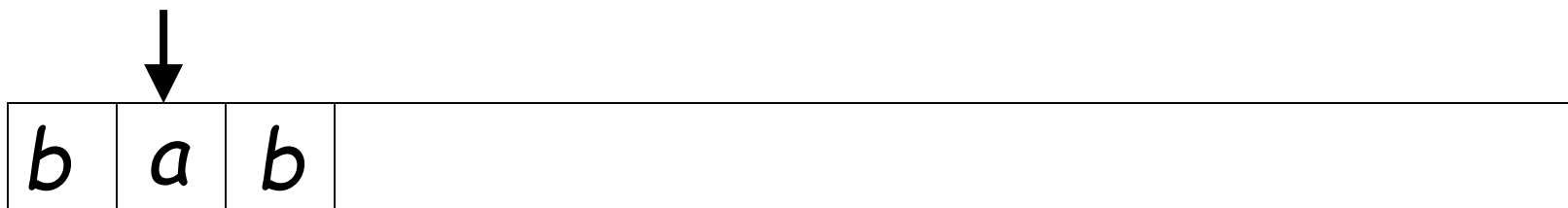
Input finished

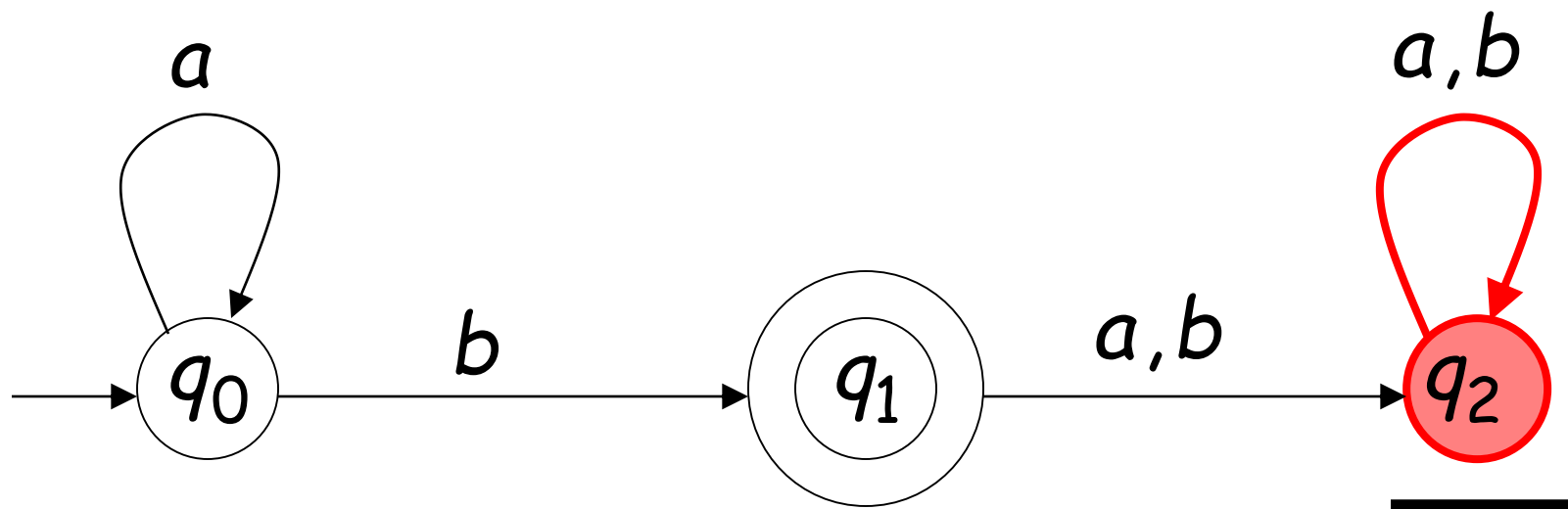


Rejection

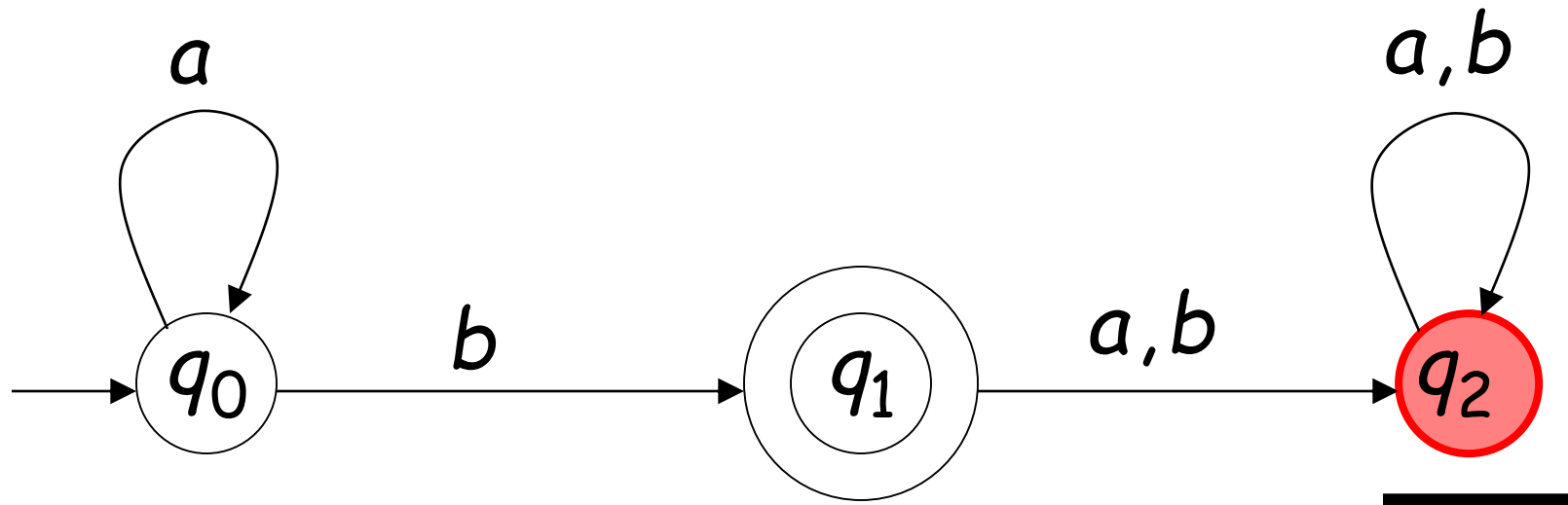
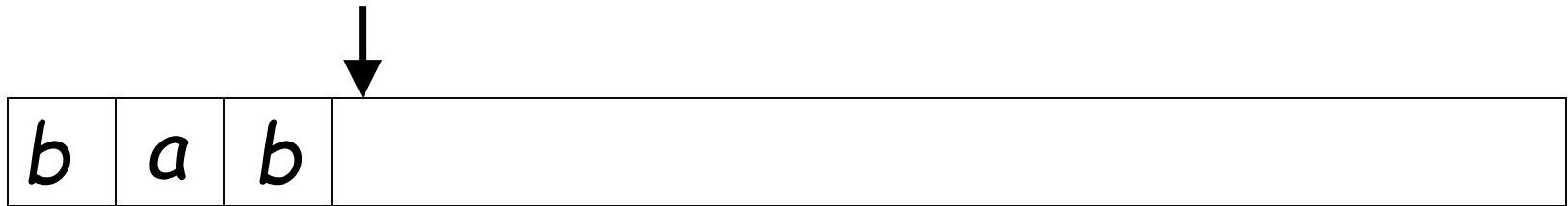








Input finished



Output: "reject"

Formal Definition

Deterministic Finite Automaton (DFA)

$$M = (Q, \Sigma, \delta, q_0, F)$$

Q : finite set of states

Σ : finite set of input alphabet

δ : transition function

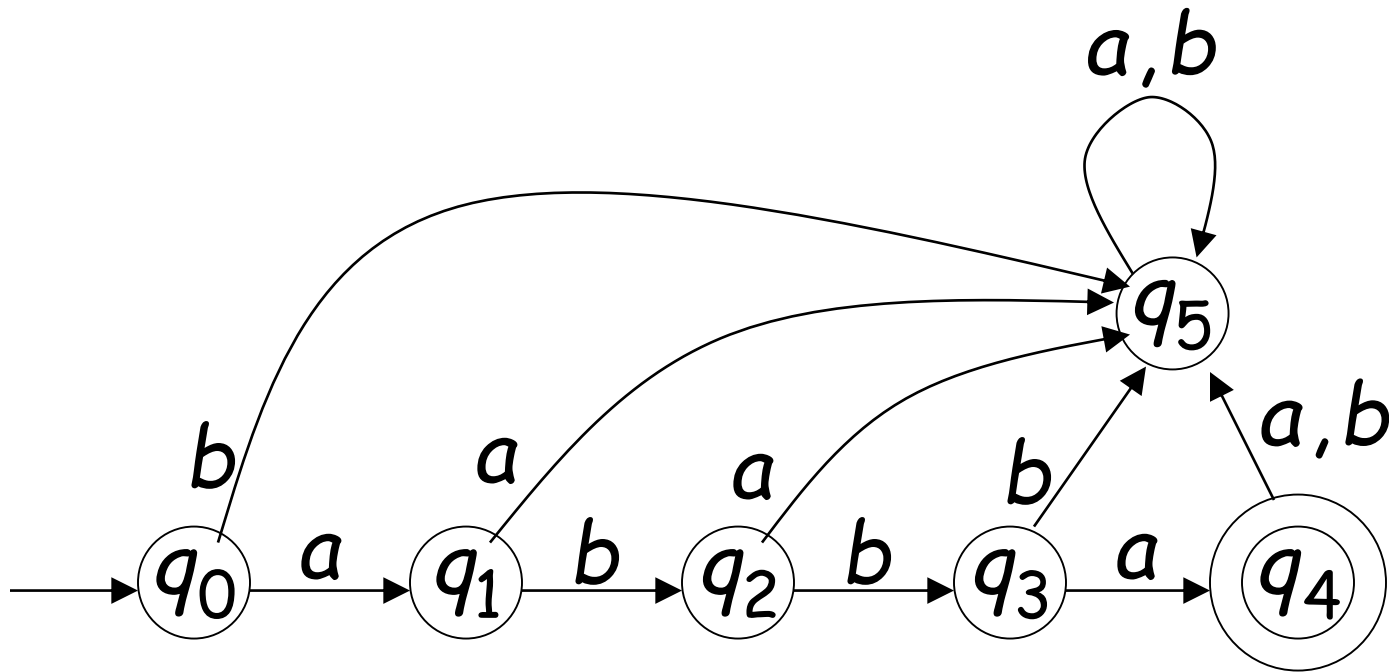
q_0 : initial state $q_0 \in Q$

F : set of final states $F \subseteq Q$

Input Alphabet Σ

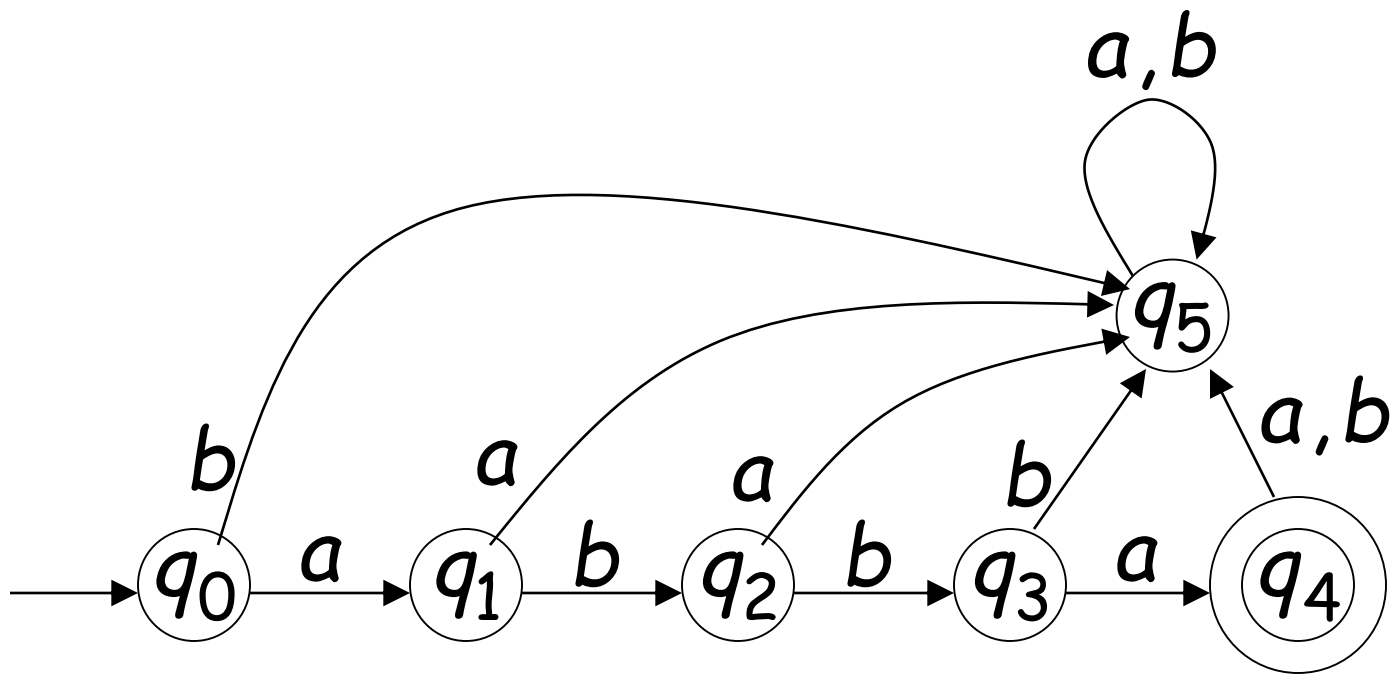
$\lambda \notin \Sigma$: the input alphabet never contains λ

$$\Sigma = \{a, b\}$$

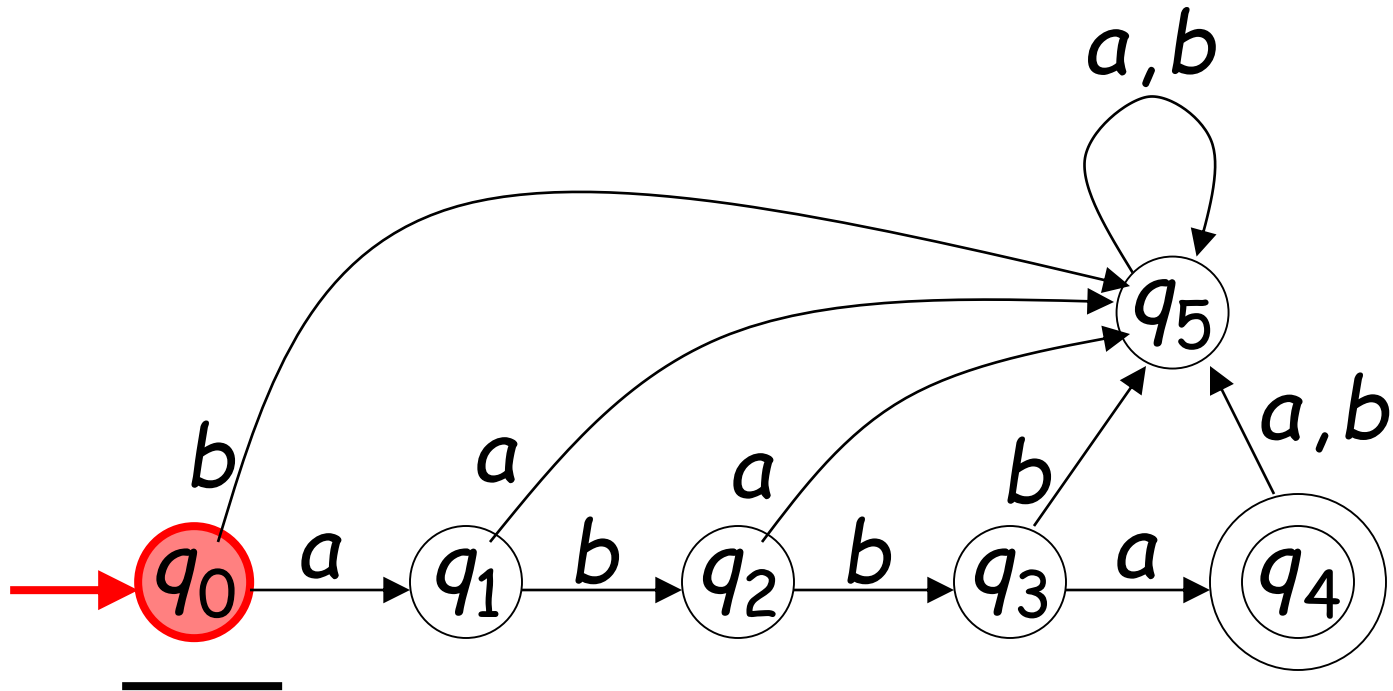


Set of States Q

$$Q = \{q_0, q_1, q_2, q_3, q_4, q_5\}$$



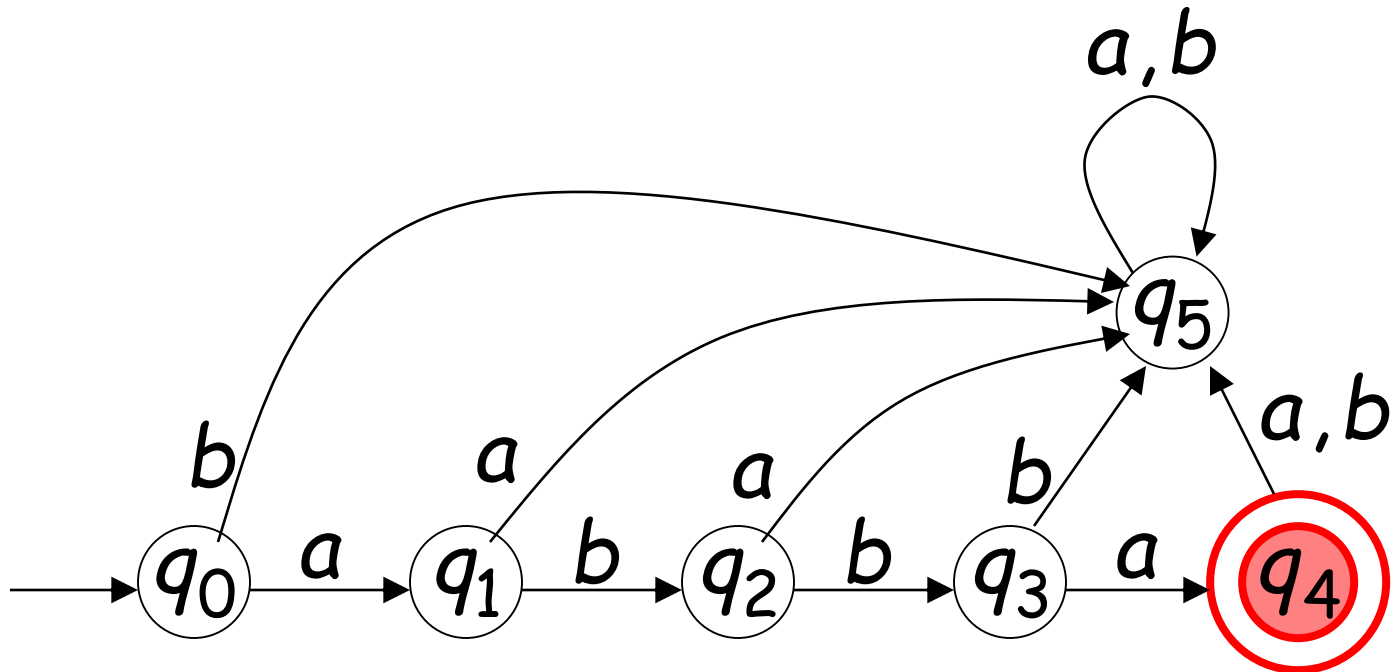
Initial State q_0



Set of Final States F

$$F = \{q_4\}$$

$$F \subseteq Q$$



To accept a string:

all the input string is scanned
and the last state (q_{final}) is accepting

$$q_{final} \in F$$

To reject a string:

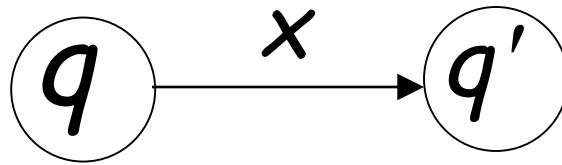
all the input string is scanned
and the last state (q_{last}) is non-accepting

$$q_{last} \notin F$$

$$q_{last} \in (Q - F)$$

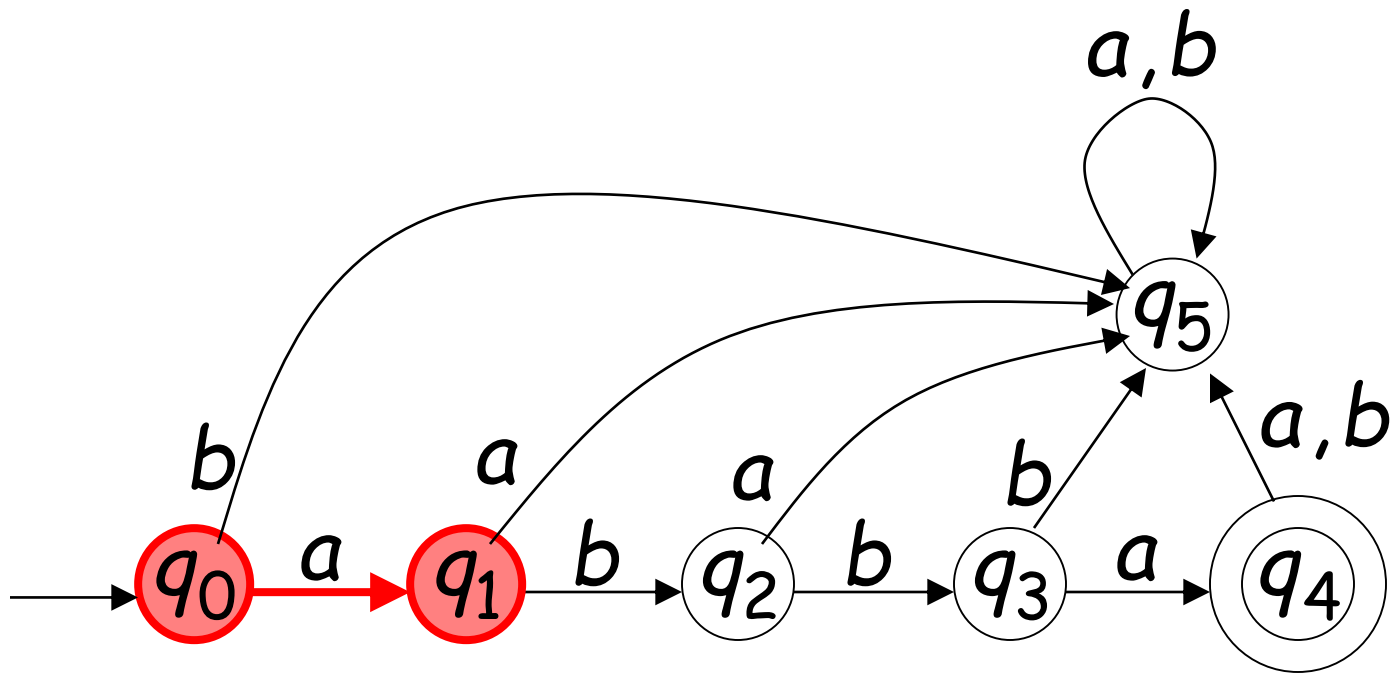
Transition Function $\delta : Q \times \Sigma \rightarrow Q$

$$\delta(q, x) = q'$$

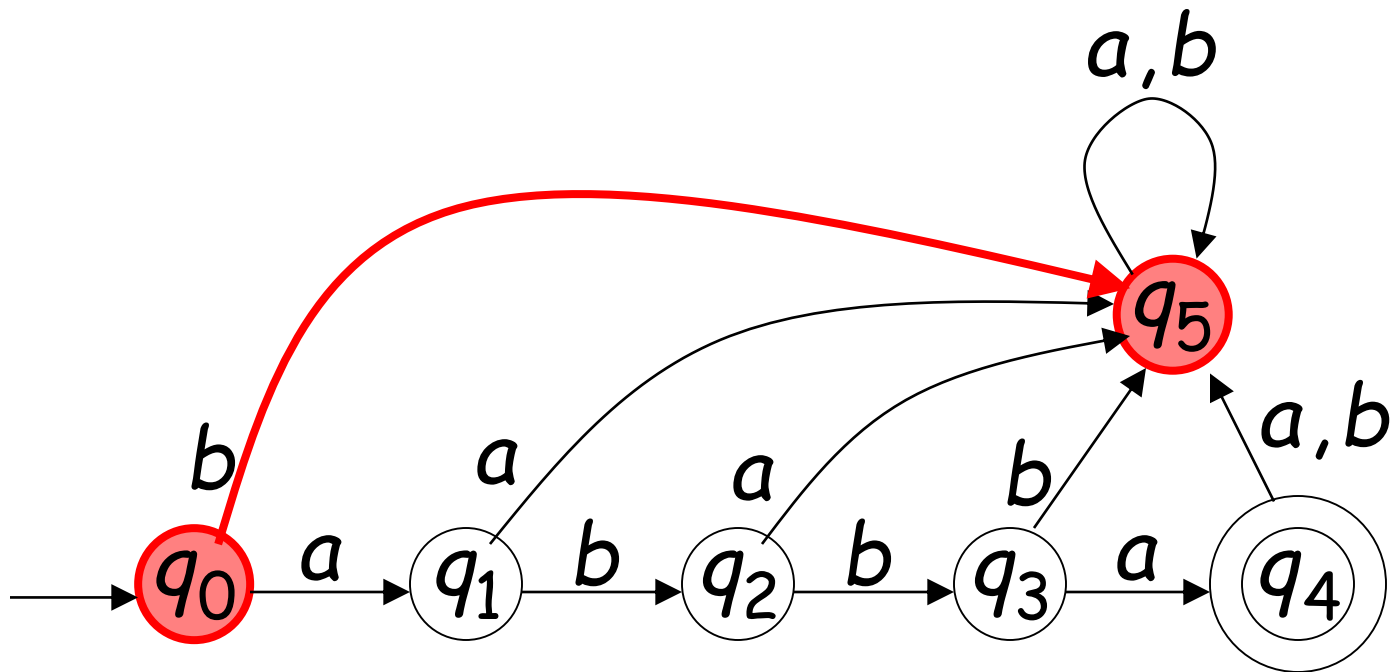


Describes the result of a transition
from state q with symbol x

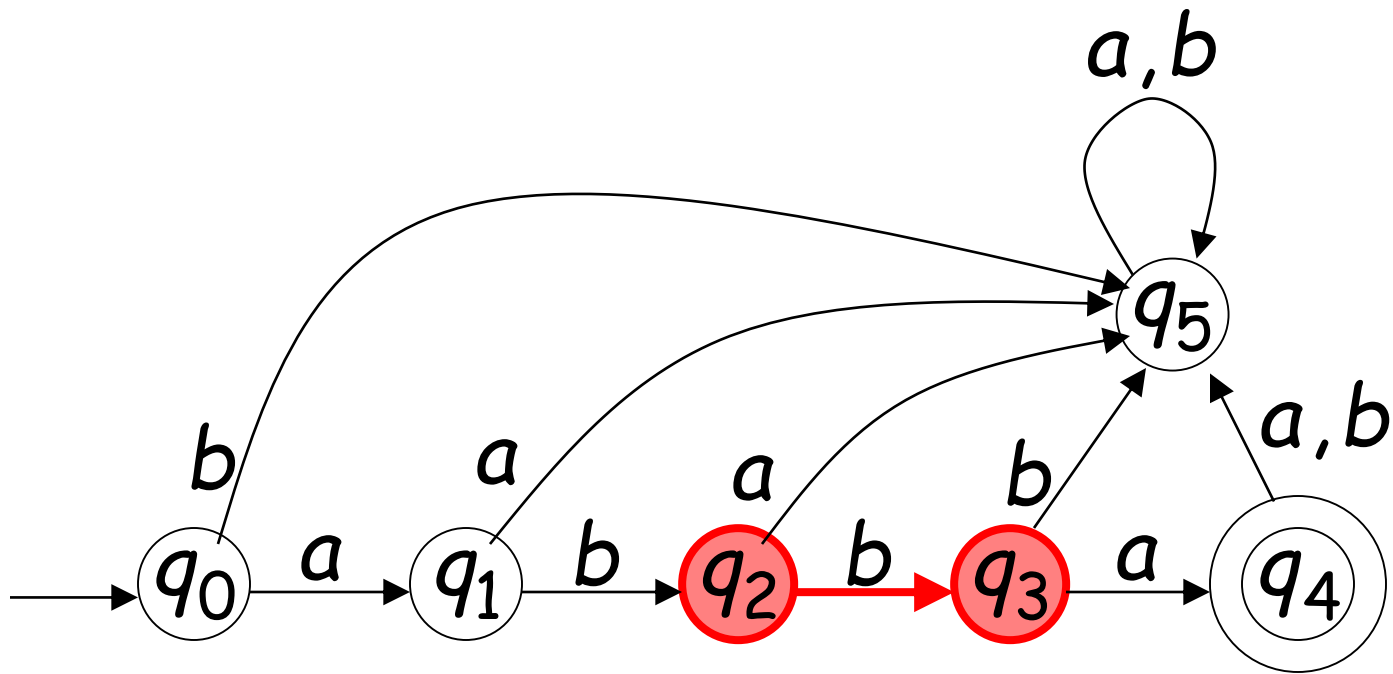
$$\delta(q_0, a) = q_1$$



$$\delta(q_0, b) = q_5$$

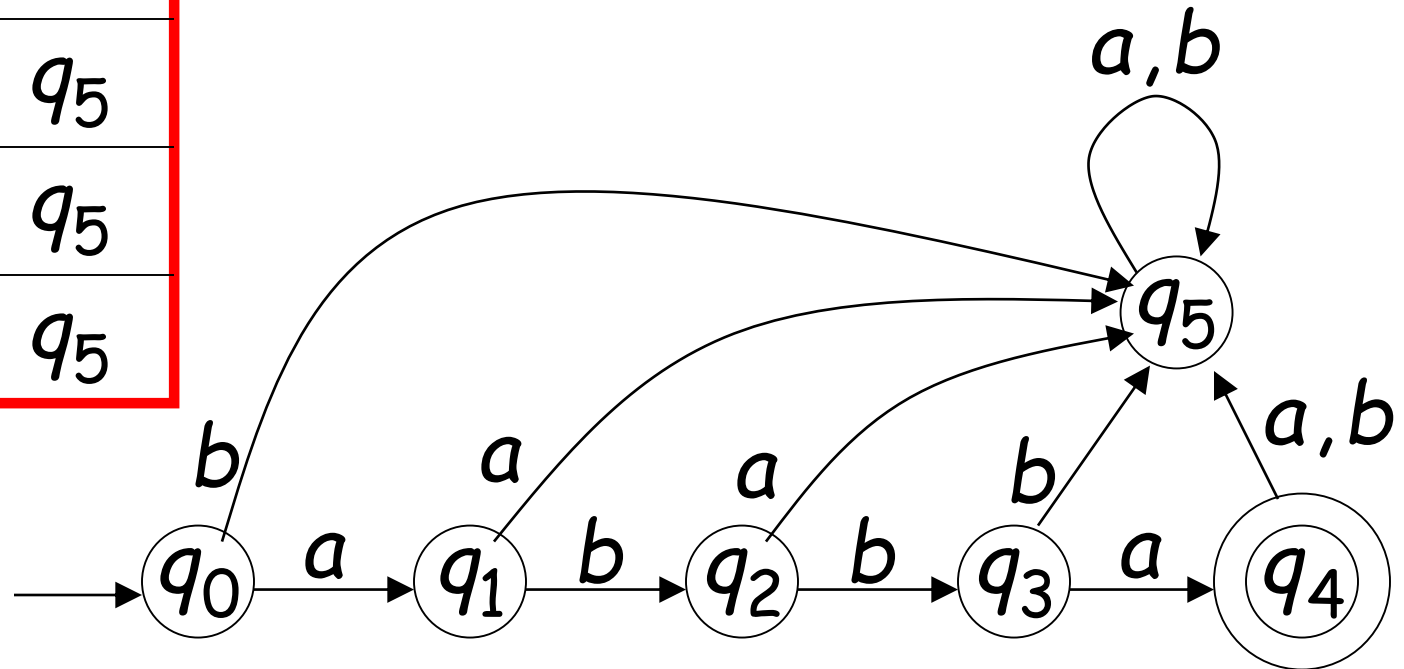


$$\delta(q_2, b) = q_3$$



Transition Function δ

δ	a	b
q_0	q_1	q_5
q_1	q_5	q_2
q_2	q_5	q_3
q_3	q_4	q_5
q_4	q_5	q_5
q_5	q_5	q_5



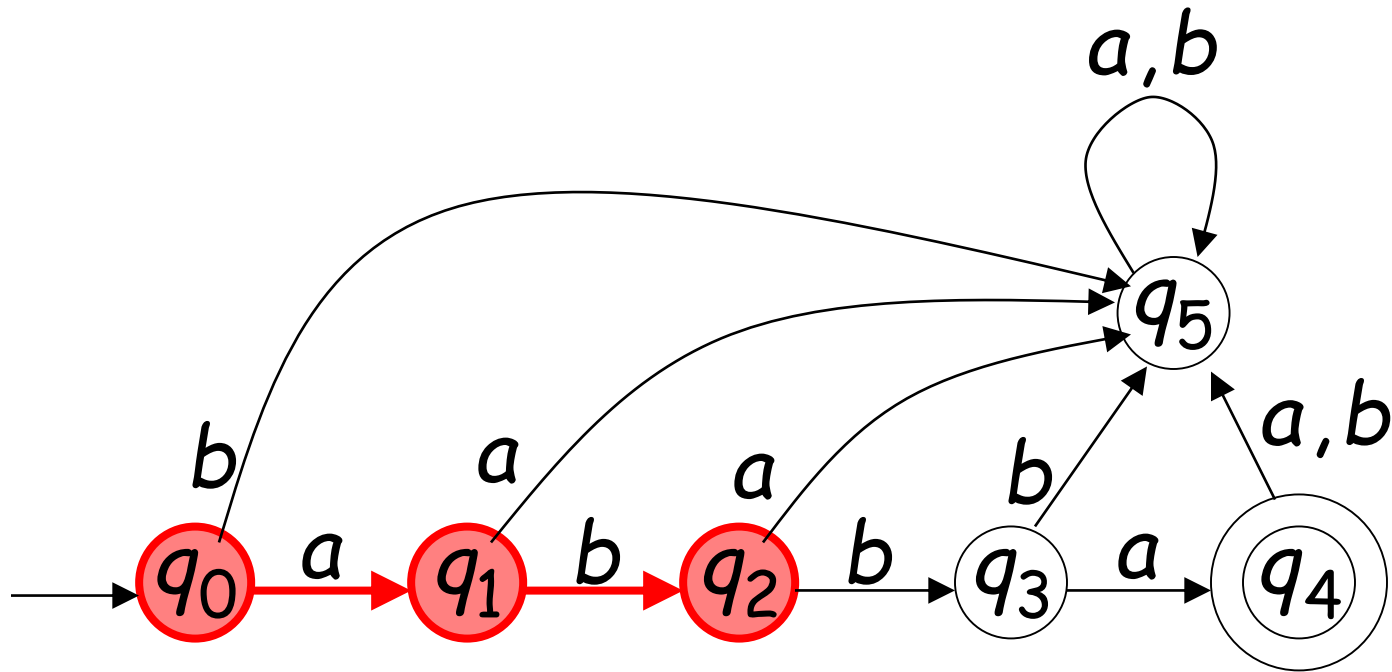
Extended Transition Function

$$\delta^* : Q \times \Sigma^* \rightarrow Q$$

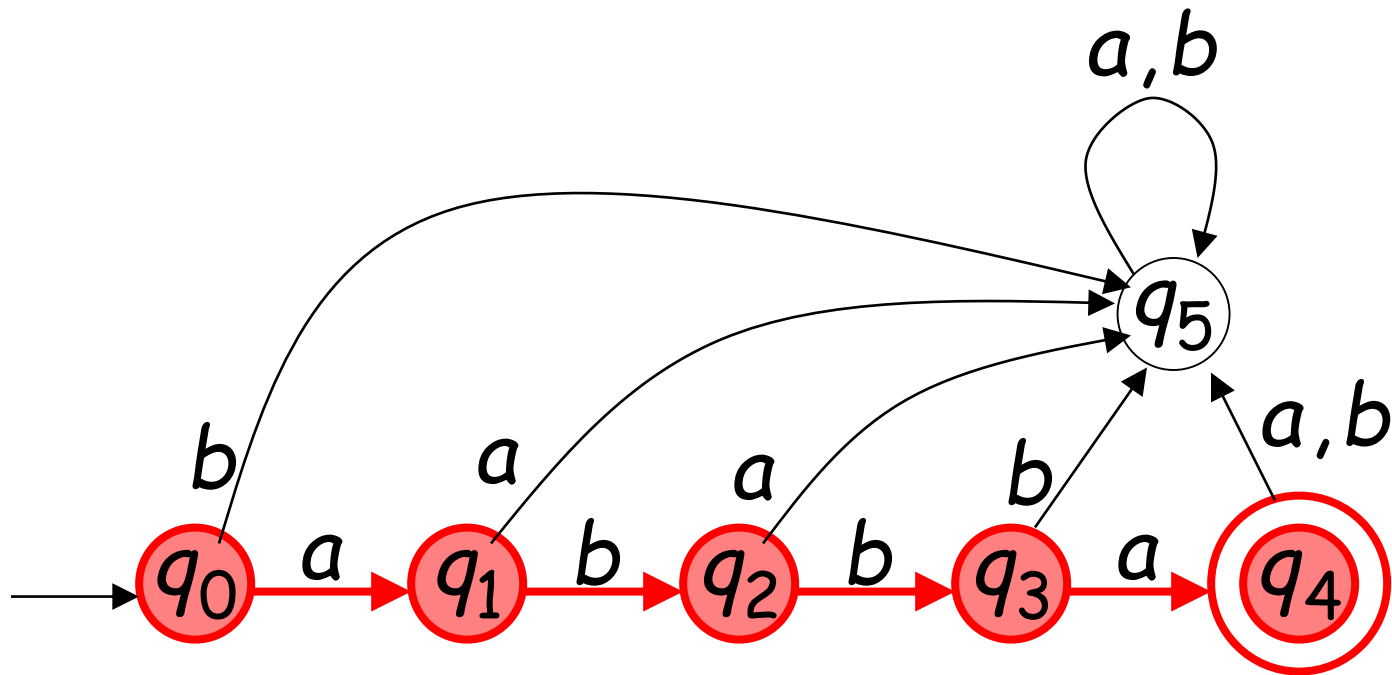
$$\delta^*(q, w) = q'$$

Describes the resulting state
after scanning string w from state q

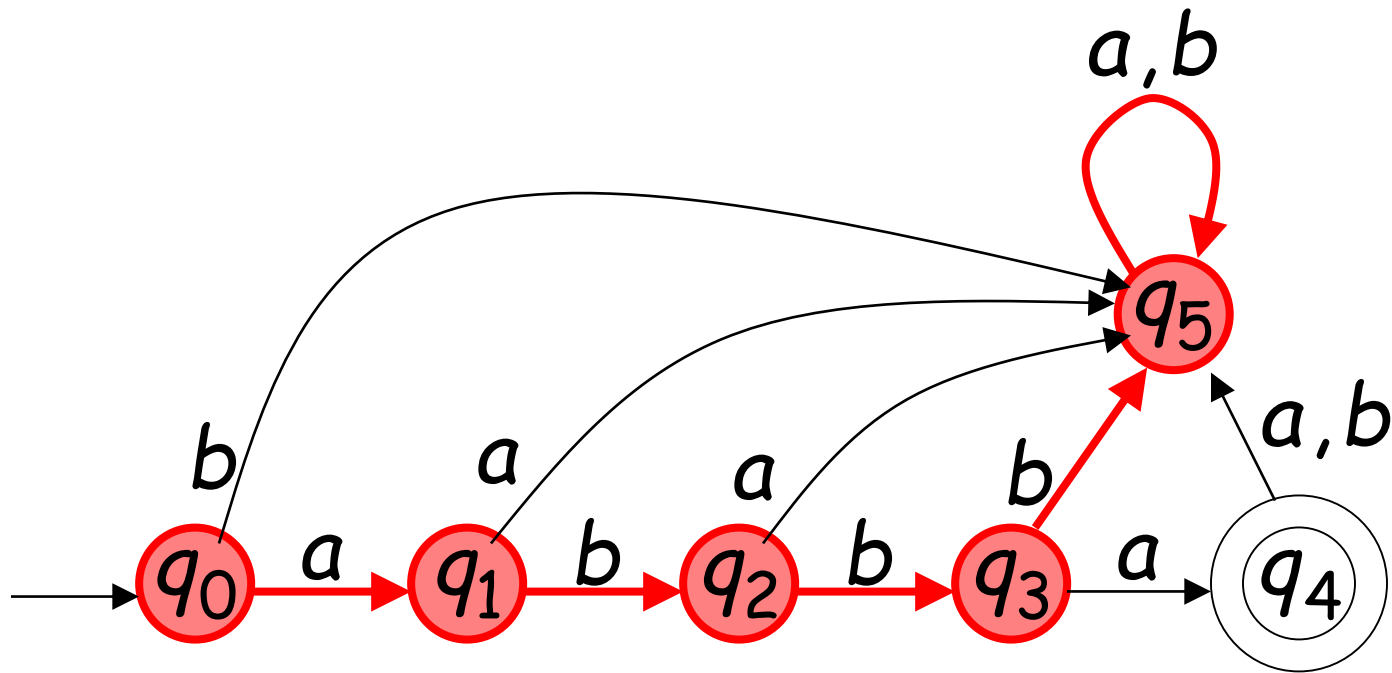
$$\delta^*(q_0, ab) = q_2$$



$$\delta^*(q_0, abba) = q_4$$



$$\delta^*(q_0, abbbaa) = q_5$$



Special case:

for any state q

$$\delta^*(q, \lambda) = q$$

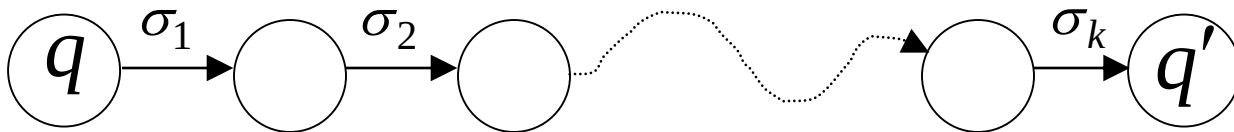
Observation: There is a walk from q to q'
with label w

$$\delta^*(q, w) = q'$$



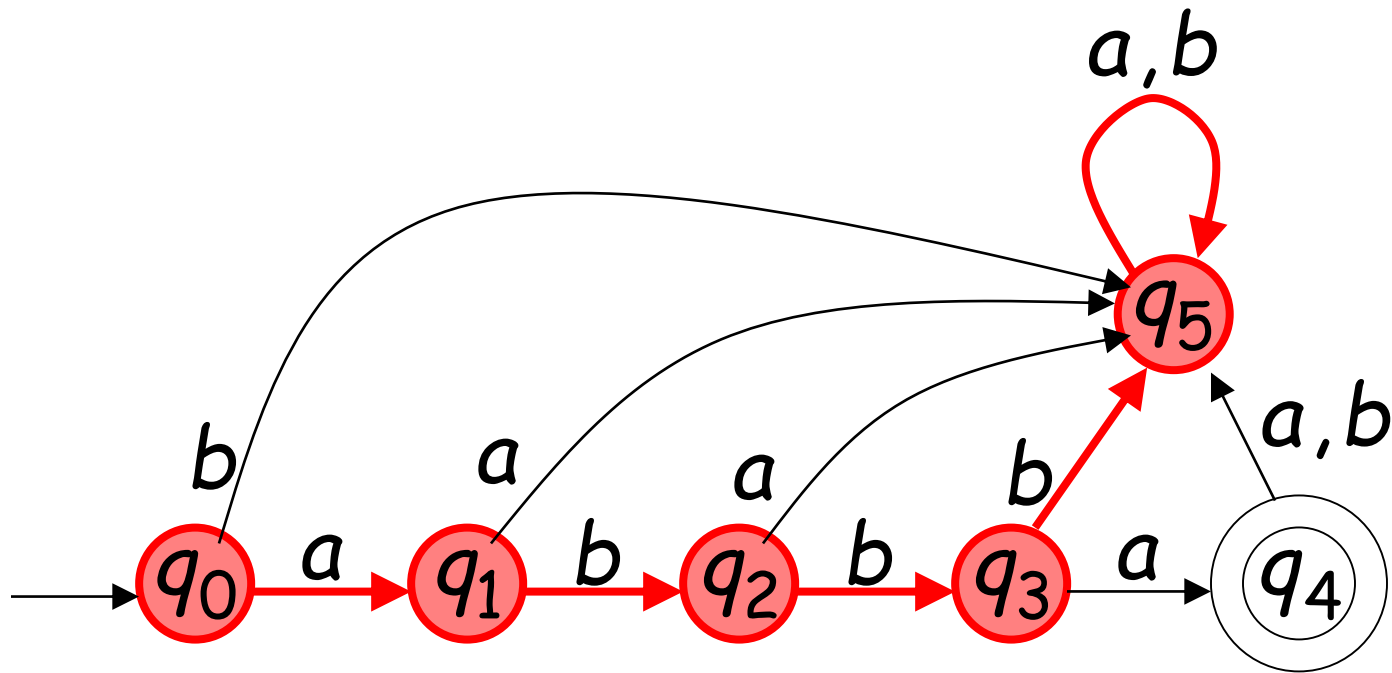
states may be repeated

$$w = \sigma_1 \sigma_2 \cdots \sigma_k$$



Example: There is a walk from q_0 to q_5
with label $abbbaa$

$$\delta^*(q_0, abbbaa) = q_5$$



Inductive Definition

Basis $\delta^*(q, \lambda) = q$

Induction $\delta^*(q, w\sigma) = \delta(\delta^*(q, w), \sigma)$



$$\begin{array}{l}
 \left. \begin{array}{l} \delta^*(q, w\sigma) = q' \\ \delta(q_1, \sigma) = q' \end{array} \right\} \Rightarrow \left. \begin{array}{l} \delta^*(q, w\sigma) = \delta(q_1, \sigma) \\ \delta^*(q, w) = q_1 \end{array} \right\} \Rightarrow \delta^*(q, w\sigma) = \delta(\delta^*(q, w), \sigma)
 \end{array}$$

δ	a	b
q_0	q_1	q_5
q_1	q_5	q_2
q_2	q_5	q_3
q_3	q_4	q_5
q_4	q_5	q_5
q_5	q_5	q_5

$$\delta^*(q_0, ab) =$$

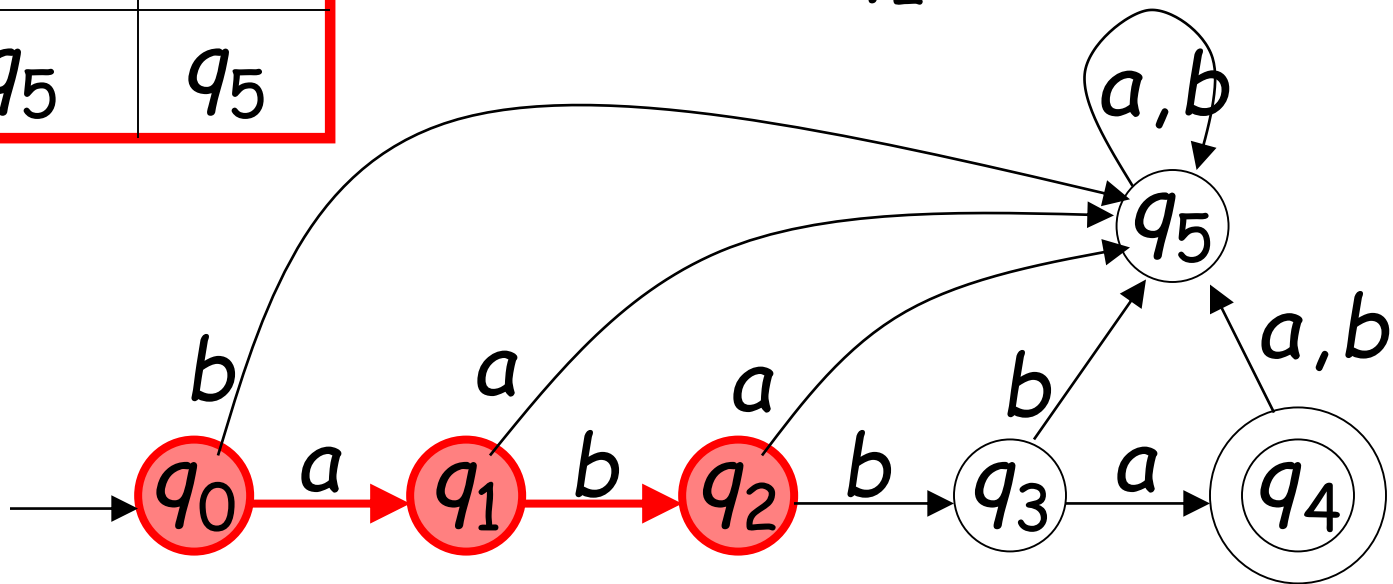
$$\delta(\delta^*(q_0, a), b) =$$

$$\delta(\delta(\delta^*(q_0, \lambda), a), b) =$$

$$\delta(\delta(q_0, a), b) =$$

$$\delta(q_1, b) =$$

$$q_2$$



δ	a	b
q_0	q_1	q_5
q_1	q_5	q_2
q_2	q_5	q_3
q_3	q_4	q_5
q_4	q_5	q_5
q_5	q_5	q_5

$$\delta^*(q, \lambda) = q$$

$$\delta^*(q_0, ab) =$$

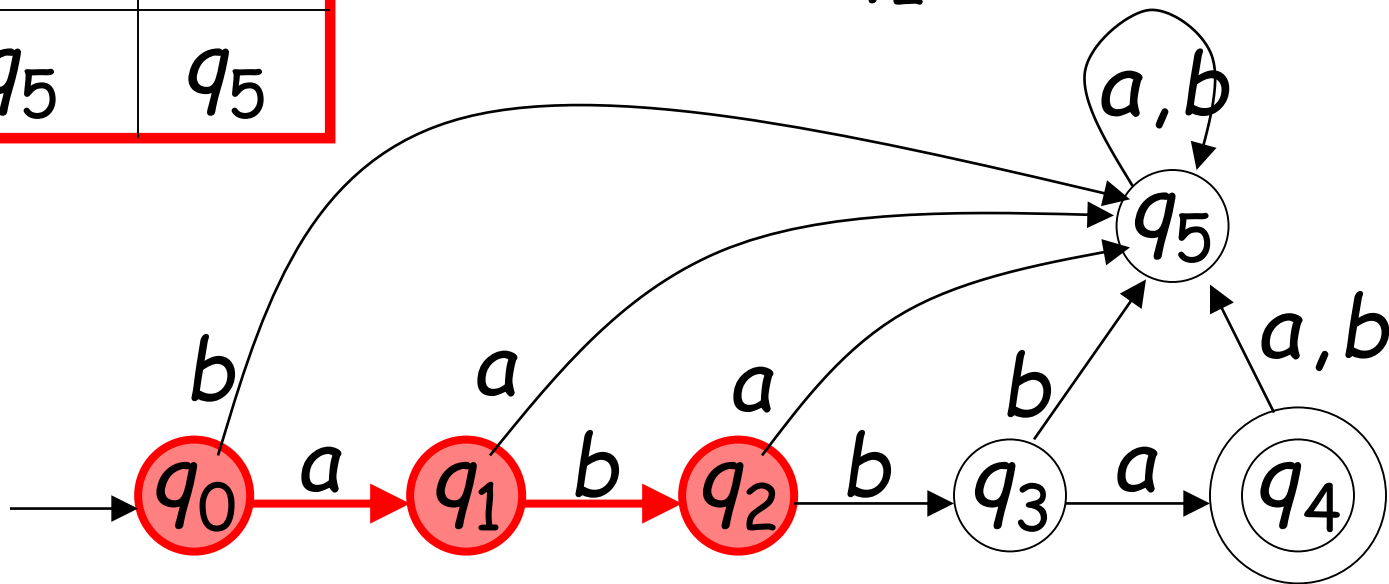
$$\delta(\delta^*(q_0, a), b) =$$

$$\delta(\delta(\delta^*(q_0, \lambda), a), b) =$$

$$\delta(\delta(q_0, a), b) =$$

$$\delta(q_1, b) =$$

$$q_2$$



δ	a	b
q_0	q_1	q_5
q_1	q_5	q_2
q_2	q_5	q_3
q_3	q_4	q_5
q_4	q_5	q_5
q_5	q_5	q_5

$$\delta^*(q_0, ab) =$$

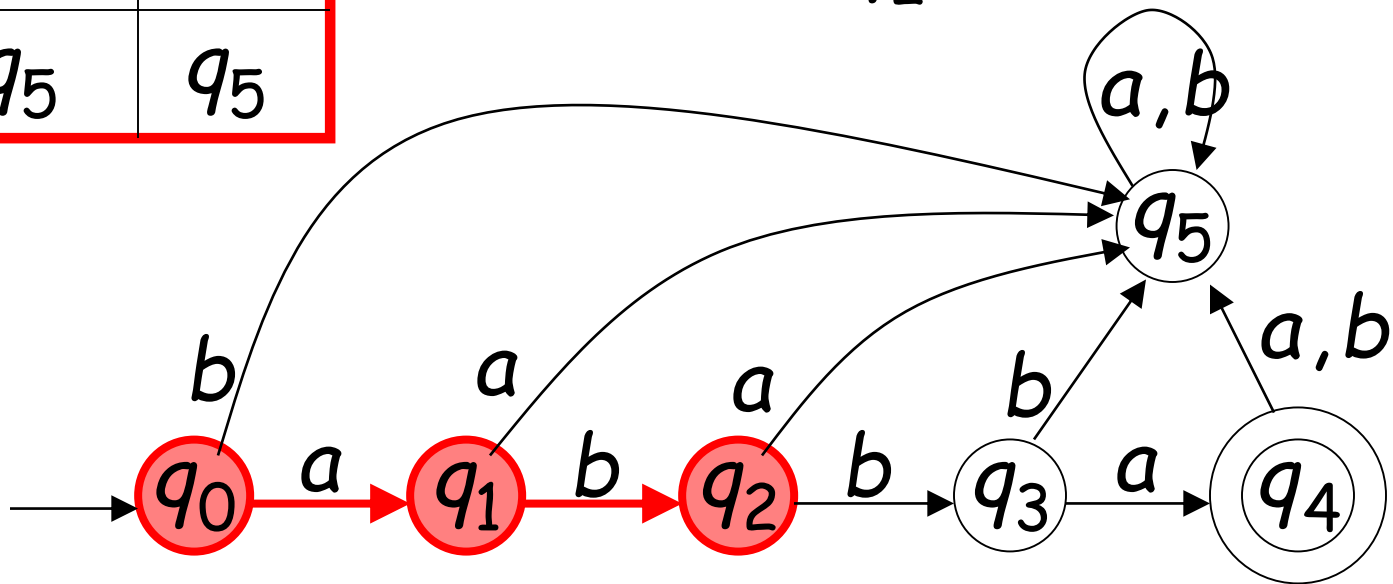
$$\delta(\delta^*(q_0, a), b) =$$

$$\delta(\delta(\delta^*(q_0, \lambda), a), b) =$$

$$\delta(\delta(q_0, a), b) =$$

$$\delta(q_1, b) =$$

$$q_2$$



δ	a	b
q_0	q_1	q_5
q_1	q_5	q_2
q_2	q_5	q_3
q_3	q_4	q_5
q_4	q_5	q_5
q_5	q_5	q_5

$$\delta^*(q_0, ab) =$$

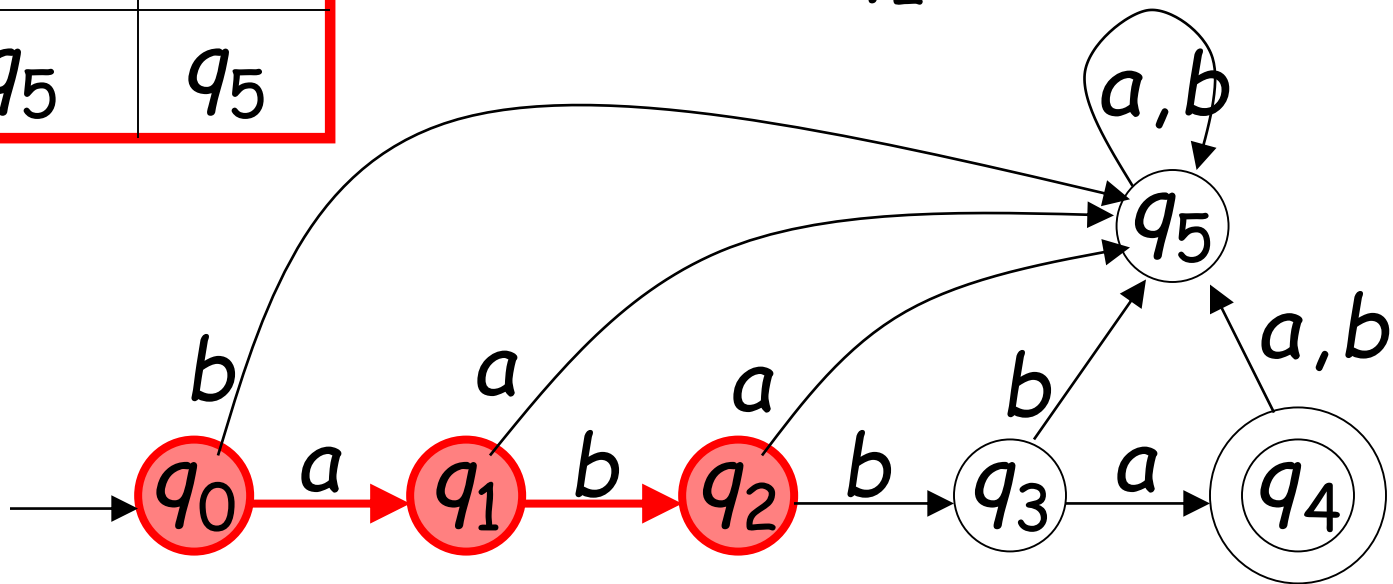
$$\delta(\delta^*(q_0, a), b) =$$

$$\delta(\delta(\delta^*(q_0, \lambda), a), b) =$$

$$\delta(\delta(q_0, a), b) =$$

$$\delta(q_1, b) =$$

$$q_2$$



δ	a	b
q_0	q_1	q_5
q_1	q_5	q_2
q_2	q_5	q_3
q_3	q_4	q_5
q_4	q_5	q_5
q_5	q_5	q_5

$$\delta^*(q_0, ab) =$$

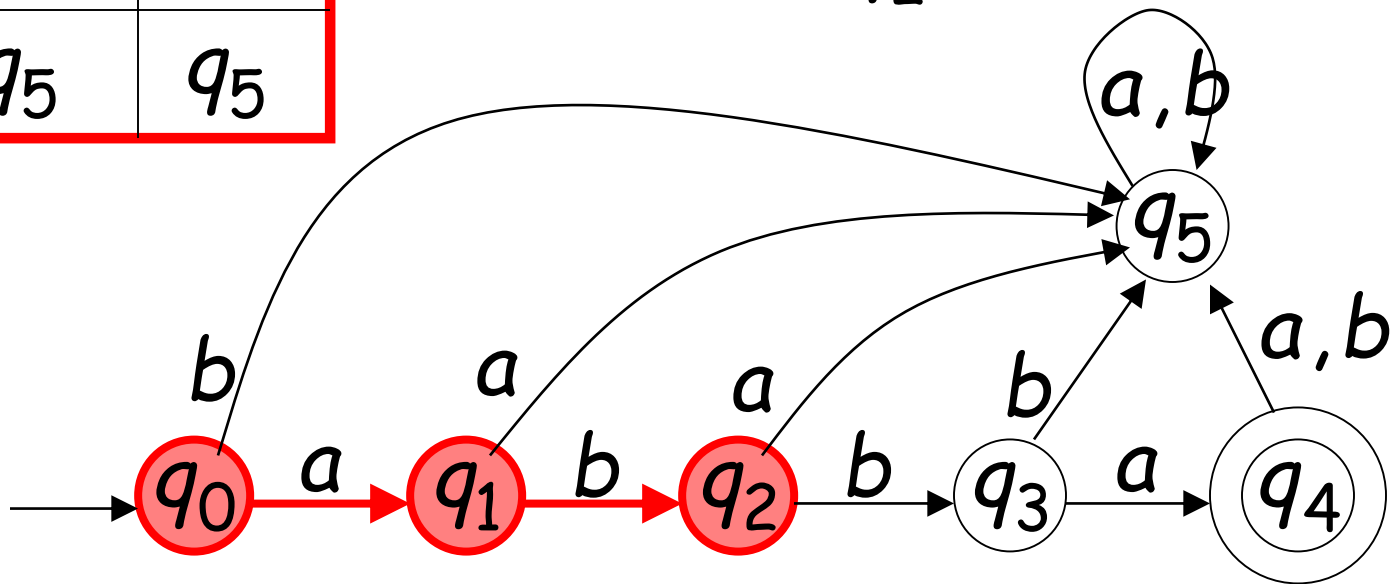
$$\delta(\delta^*(q_0, a), b) =$$

$$\delta(\delta(\delta^*(q_0, \lambda), a), b) =$$

$$\delta(\delta(q_0, a), b) =$$

$$\delta(q_1, b) =$$

$$q_2$$



δ	a	b
q_0	q_1	q_5
q_1	q_5	q_2
q_2	q_5	q_3
q_3	q_4	q_5
q_4	q_5	q_5
q_5	q_5	q_5

$$\delta^*(q_0, ab) =$$

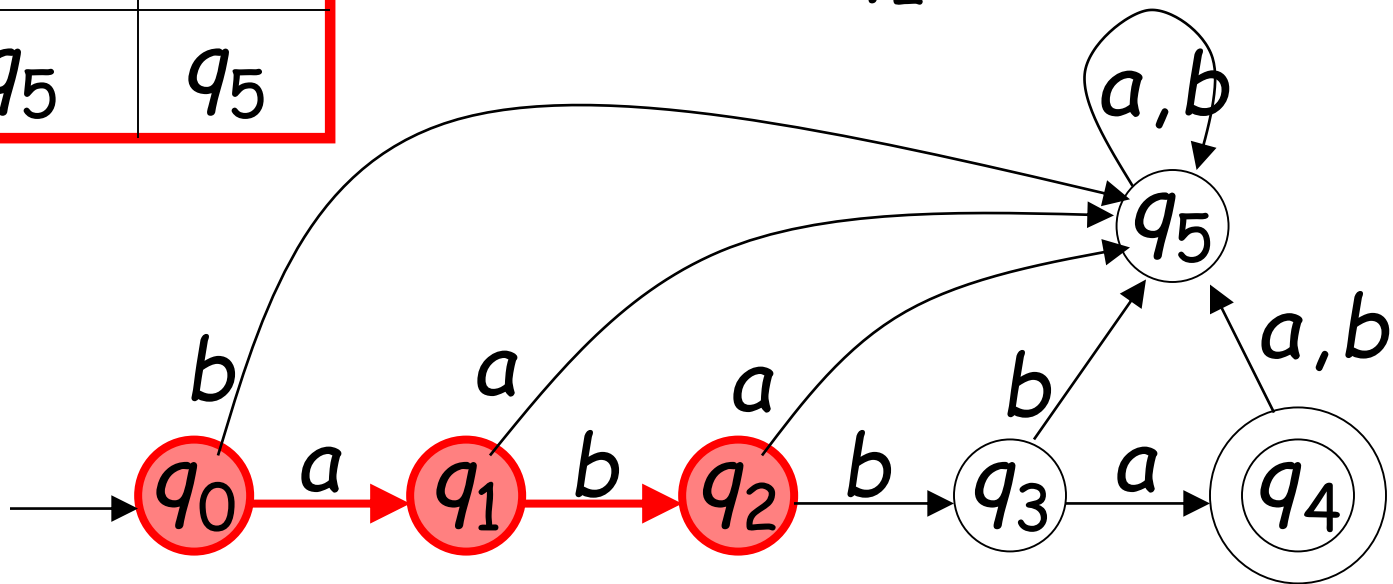
$$\delta(\delta^*(q_0, a), b) =$$

$$\delta(\delta(\delta^*(q_0, \lambda), a), b) =$$

$$\delta(\delta(q_0, a), b) =$$

$$\delta(q_1, b) =$$

q_2



δ	a	b
q_0	q_1	q_5
q_1	q_5	q_2
q_2	q_5	q_3
q_3	q_4	q_5
q_4	q_5	q_5
q_5	q_5	q_5

$$\delta^*(q_0, ab) =$$

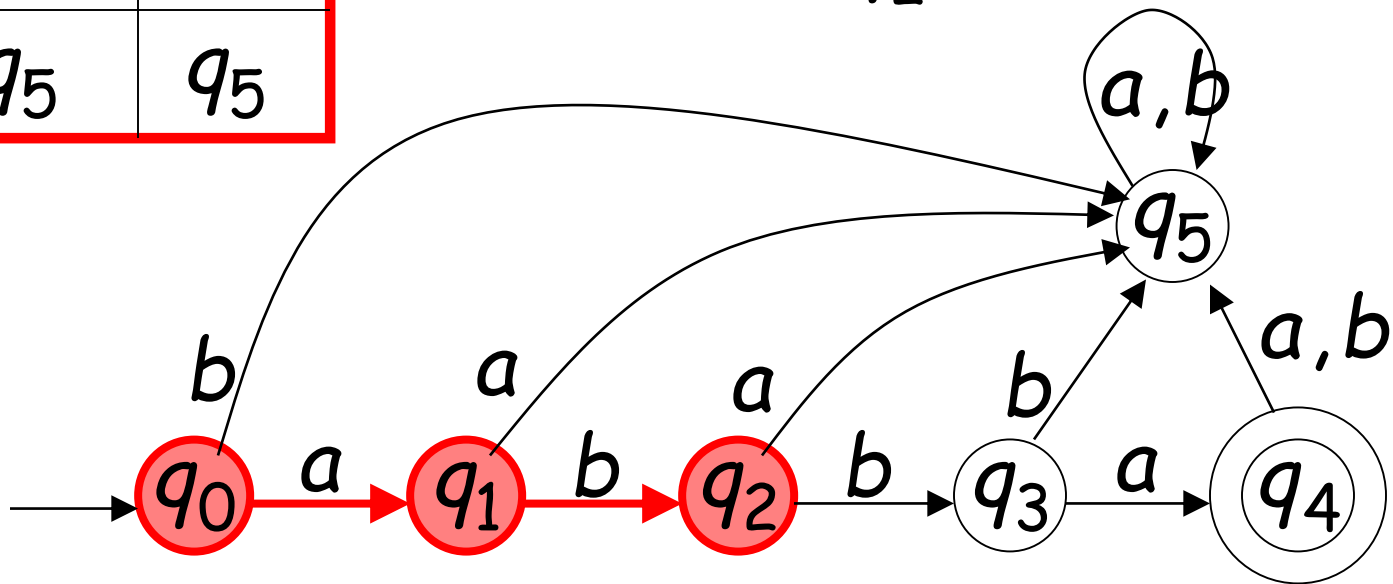
$$\delta(\delta^*(q_0, a), b) =$$

$$\delta(\delta(\delta^*(q_0, \lambda), a), b) =$$

$$\delta(\delta(q_0, a), b) =$$

$$\delta(q_1, b) =$$

$$q_2$$



Languages Accepted by DFAs

Take DFA M

Definition:

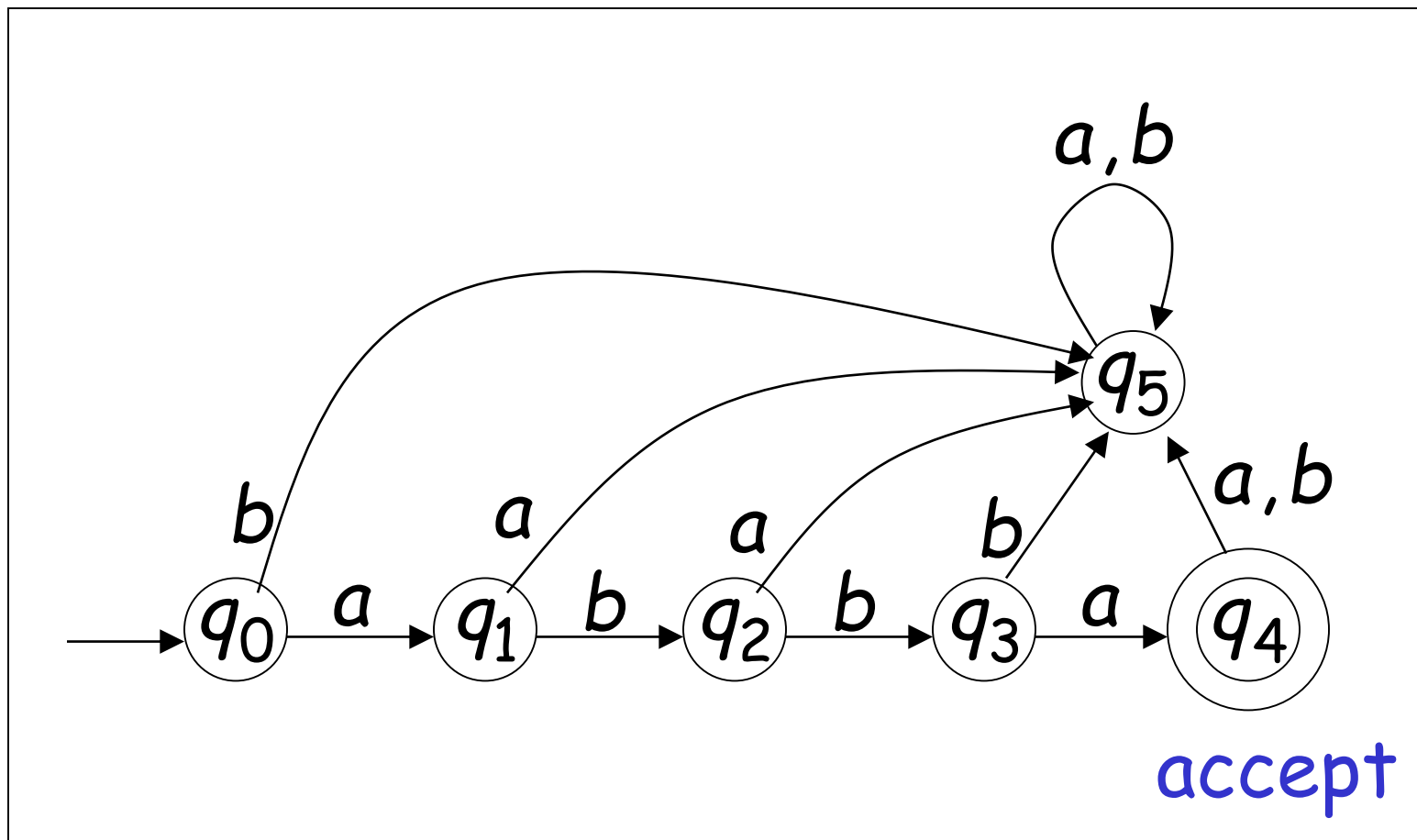
The language $L(M)$ contains
all input strings accepted by M

$$L(M) = \{ \text{strings that drive } M \text{ to a final state} \}$$

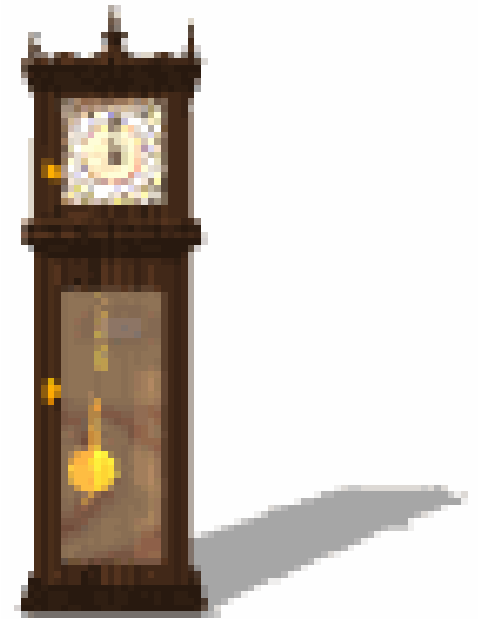
Example # 1

$$L(M) = \{abba\}$$

M



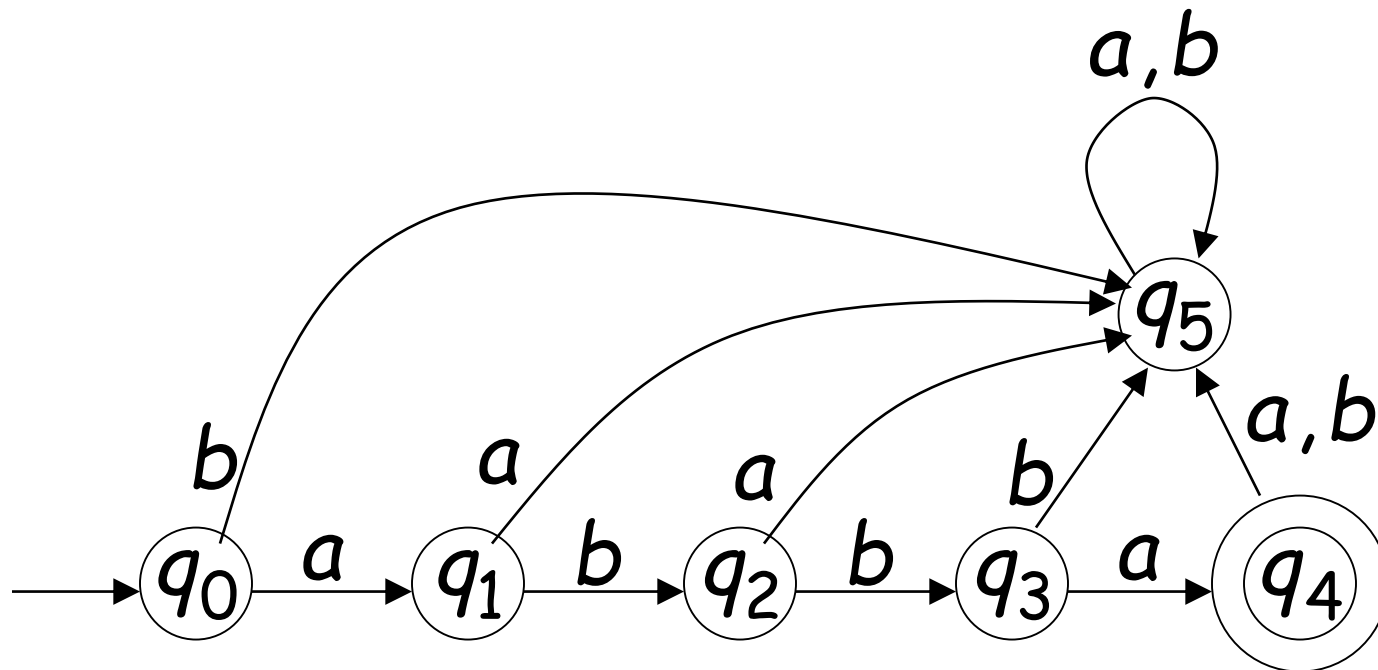
Activity Time



$$L(M) = \{\lambda, abba\}$$

This automaton accepts only one string

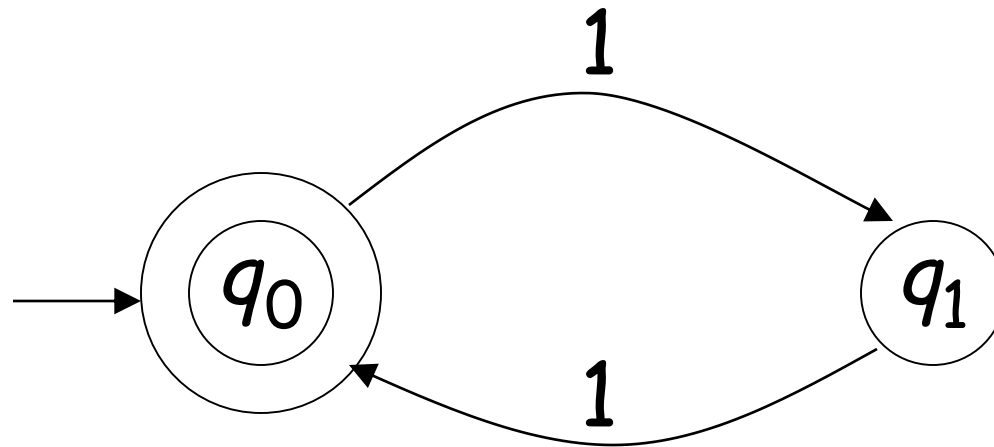
Language Accepted: $L = \{abba\}$



Make it to accept two strings $L = \{\lambda, abba\}$

Example # 2

Alphabet: $\Sigma = \{1\}$



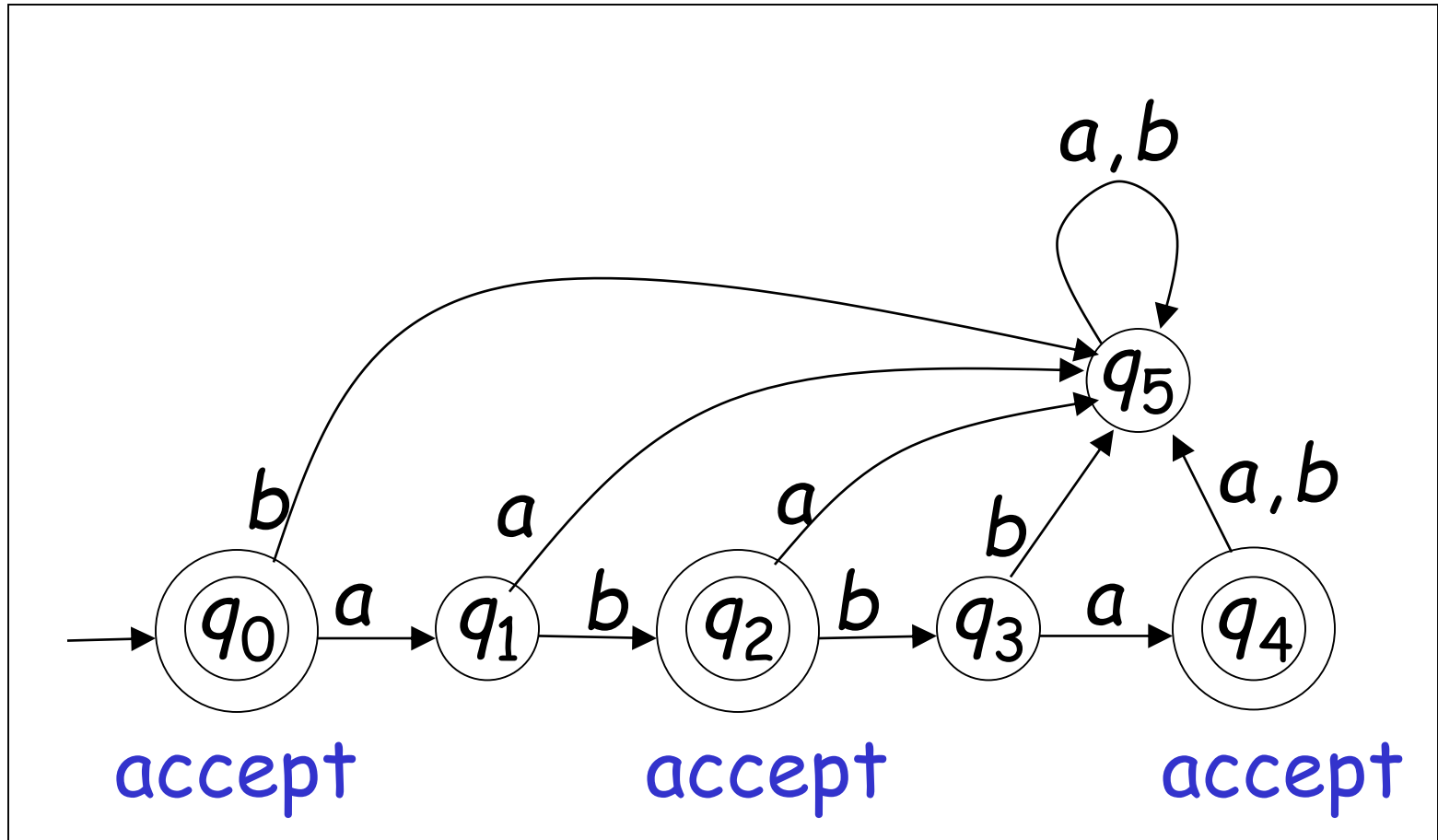
Language Accepted:

$$\begin{aligned} \text{EVEN} &= \{x : x \in \Sigma^* \text{ and } |x| \text{ is even}\} \\ &= \{\lambda, 11, 1111, 111111, \dots\} \end{aligned}$$

Example # 3

$$L(M) = \{\lambda, ab, abba\}$$

M



Languages Accepted by DFAs

Take DFA M

Definition:

The language $L(M)$ contains
all input strings accepted by M

$$L(M) = \{ \text{strings that drive } M \text{ to a final state} \}$$

Formally

For a DFA $M = (Q, \Sigma, \delta, q_0, F)$

Language accepted by M :

$$L(M) = \{w \in \Sigma^* : \delta^*(q_0, w) \in F\}$$



Observation

Language rejected by M :

$$\overline{L(M)} = \{w \in \Sigma^* : \delta^*(q_0, w) \notin F\}$$



Regular Languages

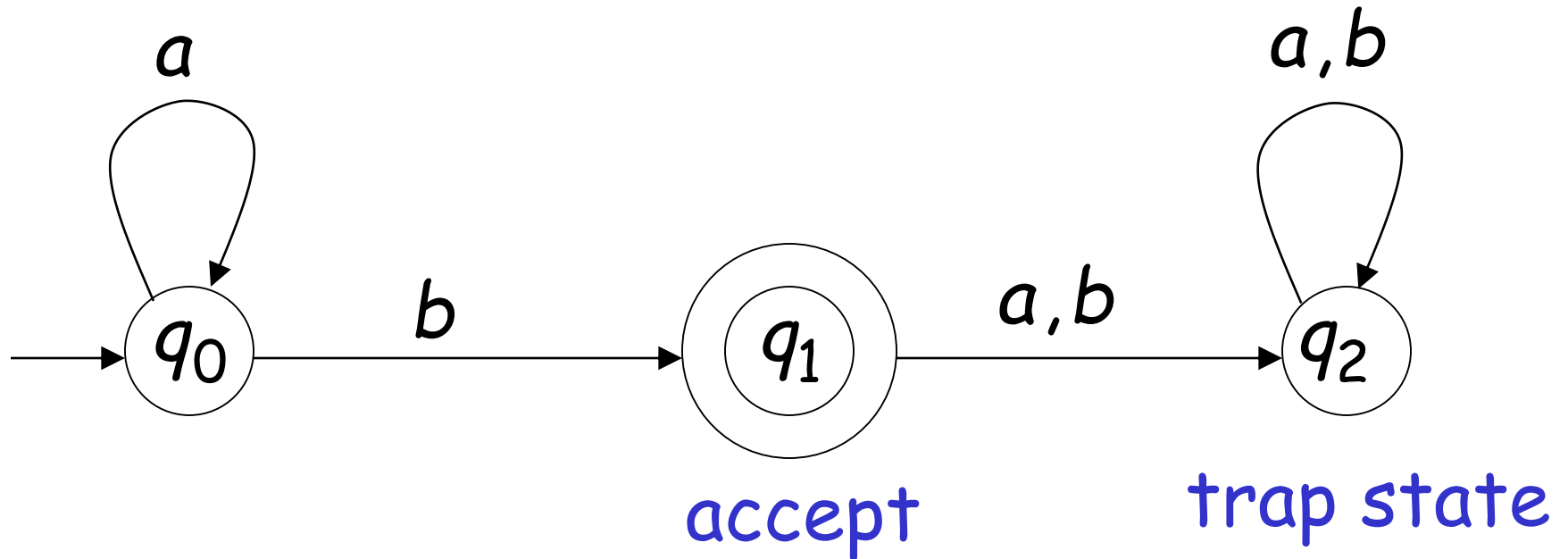
Definition:

A language L is **regular** if there is a DFA M that accepts it ($L(M) = L$)

The languages accepted by all DFAs form the family of **regular languages**

Activity # 1

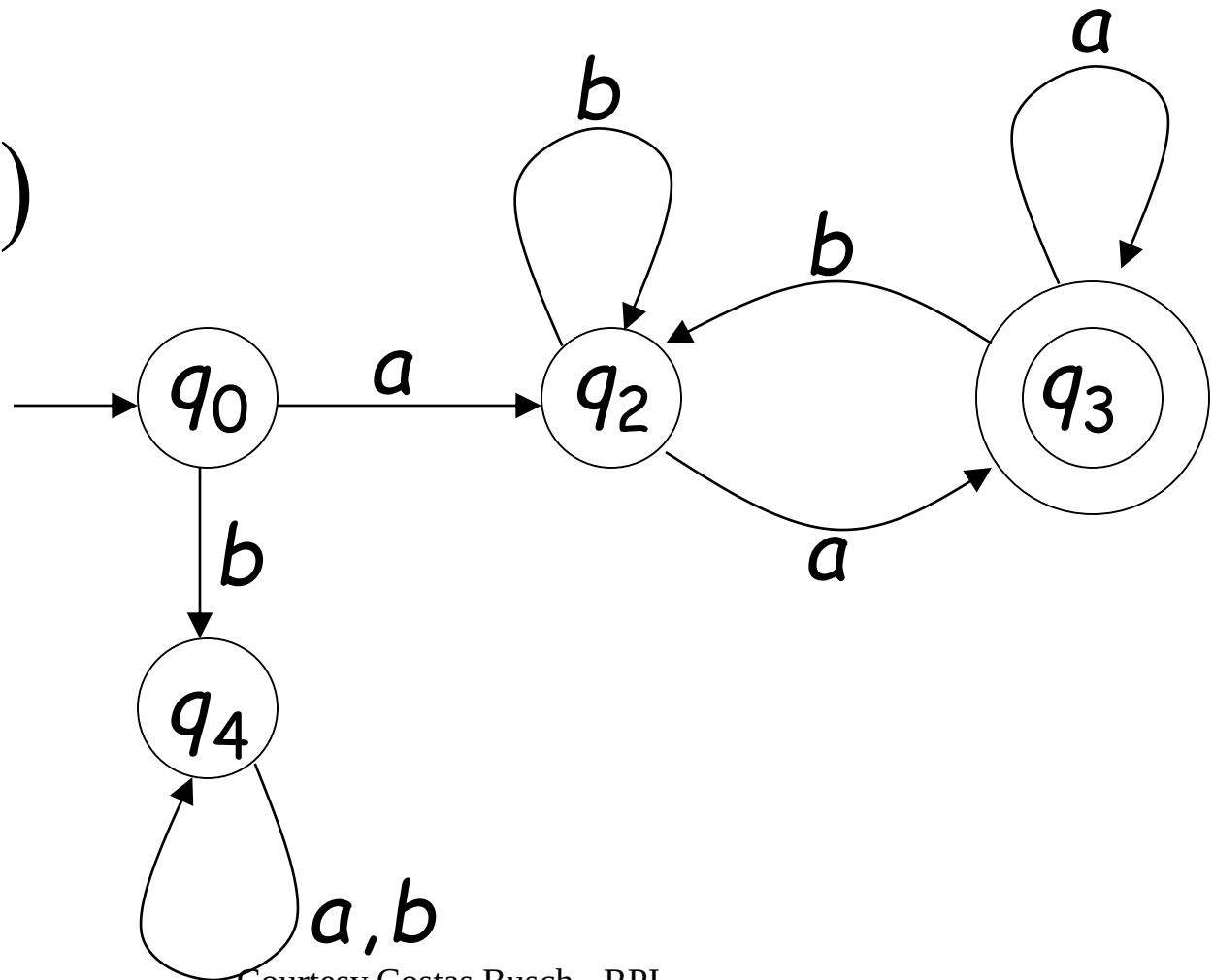
$$L(M) = \{a^n b : n \geq 0\}$$



Activity # 2

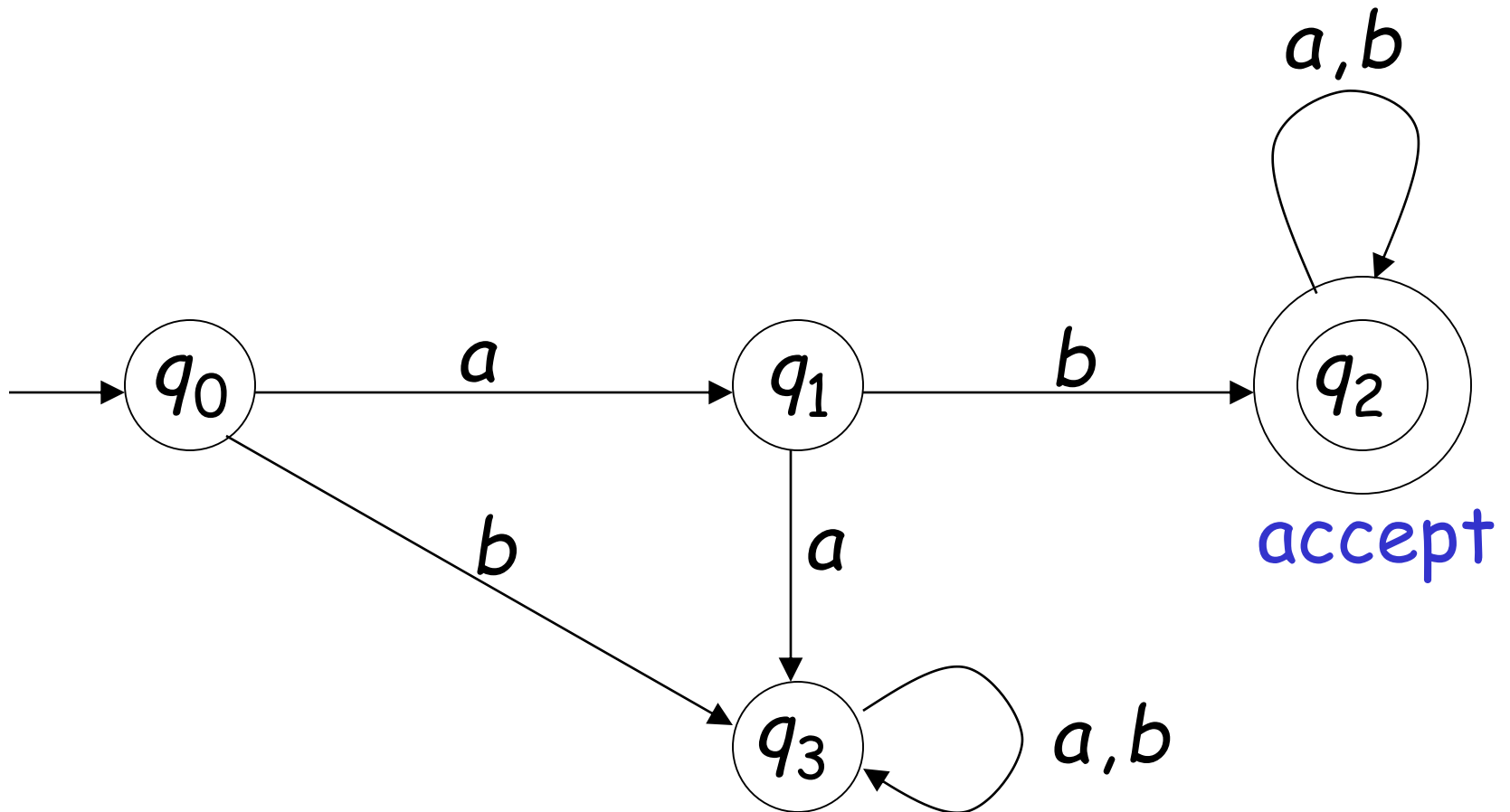
The language $L = \{awa : w \in \{a,b\}^*\}$ is regular:

$$L = L(M)$$



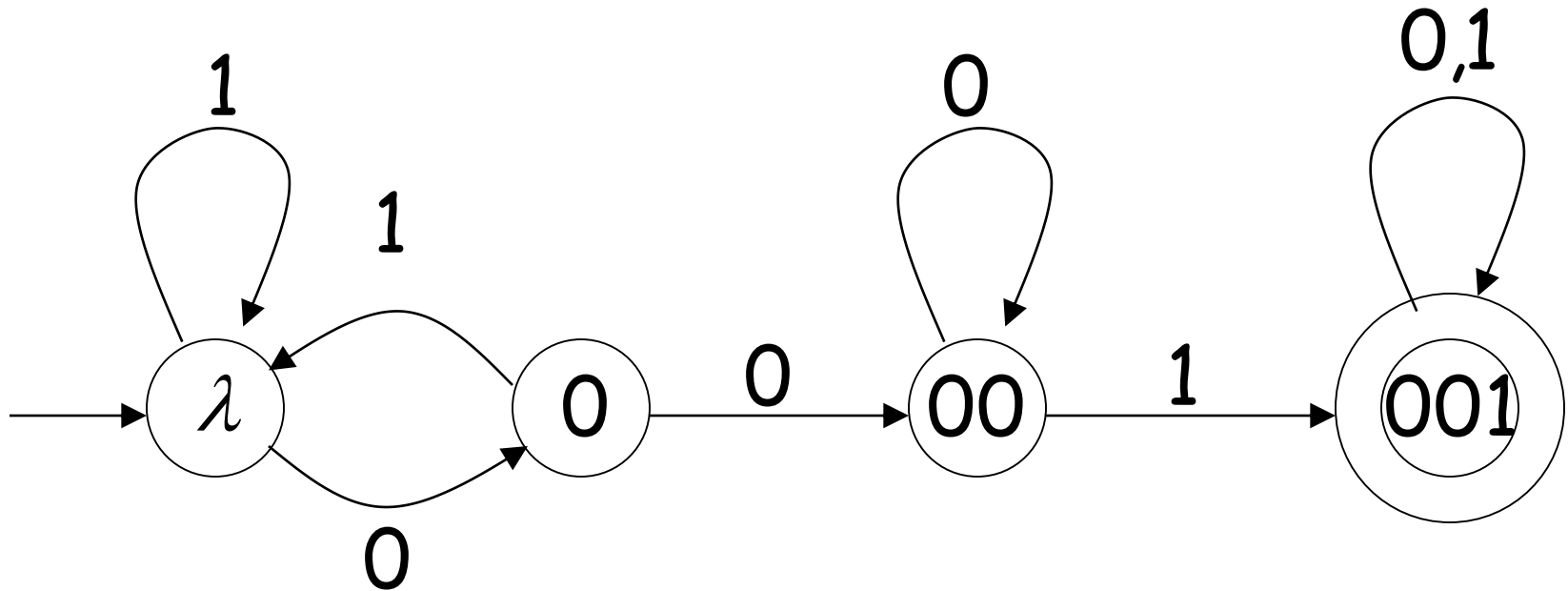
Activity # 3

$L(M) = \{ \text{all strings with prefix } ab \}$



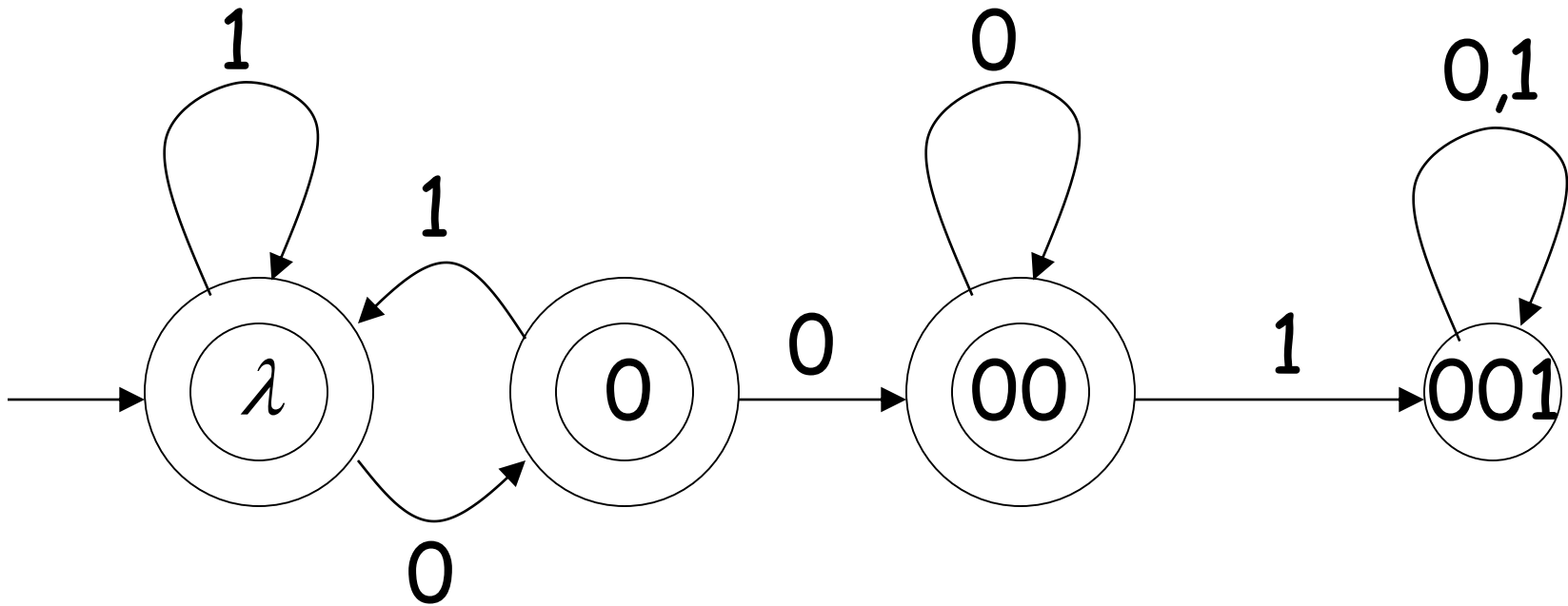
Activity # 4

$L(M) = \{ \text{all binary strings containing substring } 001 \}$



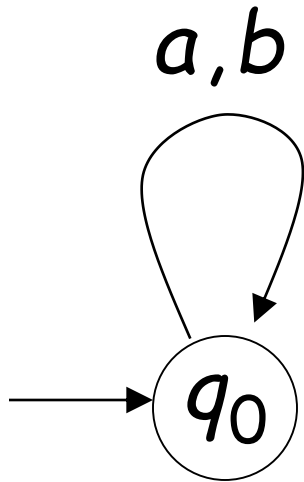
Activity # 5

$L(M) = \{ \text{all binary strings without substring } 001 \}$



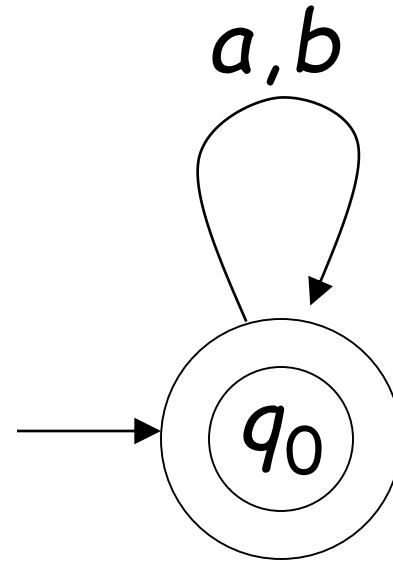
Activity # 6 & 7

$$\Sigma = \{a, b\}$$



$$L(M) = \{ \}$$

Empty language

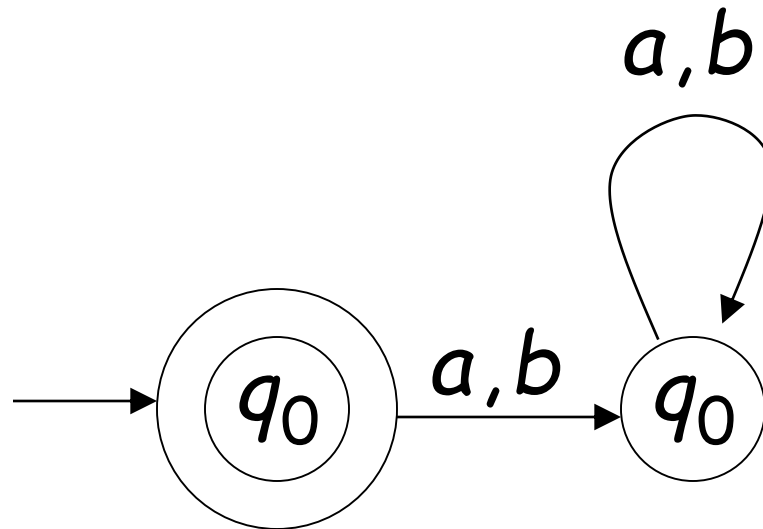


$$L(M) = \Sigma^*$$

All strings

Activity # 8

$$\Sigma = \{a, b\}$$



$$L(M) = \{\lambda\}$$

Language of the empty string

There exist languages which are not Regular:

$$L = \{a^n b^n : n \geq 0\}$$

$$\text{ADDITION} = \{x + y = z : x = 1^n, y = 1^m, z = 1^k, \\ n + m = k\}$$

There are no DFAs that accept these languages
(we will prove this in a later class)